

Planeación de acciones para robots de servicio
mediante modelos de lenguaje y sistemas basados en
reglas

Planeación de acciones para robots de servicio a
partir de instrucciones en lenguaje natural mediante
Qwen, CLIPS y modelos GPT

Luis Sergio Cano Olguin

Tutor: Dr. Jesús Savage Carmona

Agradecimientos

Resumen

Índice general

1. Introducción	1
1.1. Contexto y Motivación	2
1.2. Planteamiento del problema	3
1.3. Hipótesis	4
1.4. Objetivos	5
1.5. Estructura del documento	6
2. Antecedentes y Estado del Arte	8
2.1. Sistemas Basados en Reglas	8
2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones	9
2.2.1. Características Fundamentales de CLIPS	10
2.3. Modelos de lenguaje para interpretación de instrucciones robóticas	11
2.3.1. Familia de modelos Qwen	12
2.3.2. Modelos GPT	13
2.3.3. Interpretación semántica y generación de planes robóticos	14
2.3.4. Limitaciones de los modelos de lenguaje en robótica	15
2.3.5. Salidas estructuradas y validación	15
2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA	16
2.4. Arquitecturas neuro-simbólicas	16
2.5. Síntesis y conexión con la propuesta	19
3. Formalización del problema y arquitectura conceptual	21
3.1. Contexto arquitectónico y alcance del sistema	22
3.1.1. Integración dentro de la arquitectura ViRBot	22
3.1.2. Alcance del problema abordado	23
3.2. GPSR como dominio de referencia	23

3.2.1.	La prueba General Purpose Service Robot	23
3.2.2.	Generación de comandos y conocimiento del dominio	24
3.2.3.	Dominio de referencia y alcance lingüístico	26
3.3.	Formalización de órdenes, objetivos y planes	27
3.3.1.	Orden en lenguaje natural	28
3.3.2.	Equivalencia semántica entre formulaciones	29
3.3.3.	Entidades y conocimiento del dominio	30
3.3.4.	Representación canónica de objetivos	31
3.3.5.	Objetivos semánticos y acciones ejecutables	34
3.3.6.	Dependencias y contexto entre objetivos	35
3.4.	Arquitecturas conceptuales evaluadas	38
3.4.1.	Base común de las arquitecturas evaluadas	38
3.4.2.	Arquitectura Qwen-CLIPS	39
3.4.3.	Arquitectura de planificación directa con GPT	42
3.4.4.	Comparación conceptual de las arquitecturas	45
3.5.	Criterios conceptuales de validez	47
3.5.1.	Niveles de validez	47
3.5.2.	Validez de la interpretación semántica	48
3.5.3.	Validez y consistencia del plan	49
3.5.4.	Validez de la respuesta del sistema	50
3.5.5.	Condición de control para la etapa simbólica	50
3.5.6.	Planificación y ejecución física	51
4.	Metodología y diseño de las arquitecturas evaluadas	53
4.1.	Diseño metodológico de la investigación	54
4.1.1.	Problema experimental	54
4.1.2.	Variables y condiciones de comparación	54
4.1.3.	Estrategia general de evaluación	54
4.2.	Construcción del dominio experimental	54
4.2.1.	Fuentes de conocimiento GPSR	54
4.2.2.	Catálogos de personas, objetos y ubicaciones	54
4.2.3.	Familias de tareas	54
4.2.4.	Representación de los objetivos de referencia	54
4.3.	Construcción del conjunto de datos	54
4.3.1.	Generación de órdenes	54

4.3.2.	Generación de objetivos canónicos	54
4.3.3.	Variación lingüística	54
4.3.4.	Balance y deduplicación	54
4.3.5.	División de entrenamiento y evaluación	54
4.3.6.	Validación previa del conjunto de datos	54
4.4.	Diseño de la arquitectura Qwen-CLIPS	54
4.4.1.	Función asignada a Qwen	54
4.4.2.	Especialización para interpretación semántica	54
4.4.3.	Interfaz entre objetivos y planificación simbólica	54
4.4.4.	Función asignada a CLIPS	54
4.5.	Diseño de la arquitectura GPT directo	54
4.5.1.	Funciones asignadas al modelo	54
4.5.2.	Conocimiento y restricciones proporcionadas	54
4.5.3.	Contrato de salida	54
4.5.4.	Plan ejecutable, aclaración y rechazo	54
4.6.	Diseño de la comparación	54
4.6.1.	Igualdad del dominio de evaluación	54
4.6.2.	Igualdad del repertorio de acciones	54
4.6.3.	Separación entre interpretación y planificación	54
5.	Implementación	55
5.1.	Plataforma experimental	56
5.1.1.	Robot Justina	56
5.1.2.	Plataforma de cómputo	56
5.1.3.	Entorno de software y ROS 2	56
5.2.	Implementación del conjunto de datos	56
5.2.1.	Generador de órdenes	56
5.2.2.	Catálogos y CompetitionTemplate	56
5.2.3.	Validadores	56
5.2.4.	Dataset final	56
5.3.	Entrenamiento e inferencia de Qwen2.5-7B	56
5.3.1.	Preparación del modelo	56
5.3.2.	Configuración QLoRA	56
5.3.3.	Hiperparámetros de entrenamiento	56
5.3.4.	Selección del checkpoint	56

5.3.5.	Inferencia	56
5.3.6.	Despliegue del modelo	56
5.4.	Implementación del planificador CLIPS	56
5.4.1.	Conversión de objetivos a hechos gpsr-goal	56
5.4.2.	Representación del estado simbólico	56
5.4.3.	Reglas de expansión	56
5.4.4.	Generación de acciones	56
5.4.5.	Condición de finalización	56
5.5.	Integración Qwen-CLIPS con Justina	56
5.5.1.	Componentes ROS 2	56
5.5.2.	Flujo de comunicación	56
5.5.3.	Comunicación con el ejecutor	56
5.6.	Implementación del planificador GPT	56
5.6.1.	Interfaz con la API	56
5.6.2.	System prompt	56
5.6.3.	JSON Schema y Structured Outputs	56
5.6.4.	Generación de objetivos y planes	56
5.6.5.	Tratamiento de aclaraciones y rechazos	56
5.6.6.	Validación determinista	56
5.7.	Reproducibilidad	56
5.7.1.	Versiones de modelos	56
5.7.2.	Configuraciones experimentales	56
5.7.3.	Registro de ejecuciones	56
6.	Validación y experimentación	57
6.1.	Diseño experimental	58
6.1.1.	Conjunto de prueba	58
6.1.2.	Procedimiento experimental	58
6.1.3.	Repeticiones y control de variabilidad	58
6.2.	Condiciones experimentales	58
6.2.1.	Arquitectura Qwen-CLIPS	58
6.2.2.	Arquitectura GPT directo	58
6.2.3.	Condición de control con objetivos de referencia y CLIPS	58
6.3.	Evaluación de la interpretación semántica	58
6.3.1.	Exactitud de objetivos	58

6.3.2.	Validez estructural	58
6.3.3.	Exactitud de entidades y parámetros	58
6.3.4.	Dependencias entre objetivos	58
6.4.	Evaluación de la planificación	58
6.4.1.	Acciones válidas	58
6.4.2.	Consistencia del plan	58
6.4.3.	Cumplimiento de subtareas	58
6.4.4.	Precondiciones y dependencias	58
6.4.5.	Planificabilidad	58
6.5.	Clasificación de errores	58
6.5.1.	Errores de interpretación	58
6.5.2.	Errores de planificación	58
6.5.3.	Errores combinados	58
6.6.	Evaluación física con Justina	58
6.6.1.	Selección de tareas	58
6.6.2.	Procedimiento de ejecución	58
6.6.3.	Criterios de éxito y fallo	58
6.7.	Rendimiento computacional	58
6.7.1.	Latencia	58
6.7.2.	Uso de recursos	58
6.7.3.	Tokens y coste del modelo GPT	58
7.	Resultados	59
7.1.	Resultados de la especialización de Qwen	60
7.1.1.	Comportamiento durante entrenamiento	60
7.1.2.	Evaluación del modelo seleccionado	60
7.2.	Resultados de interpretación semántica	60
7.2.1.	Qwen	60
7.2.2.	GPT	60
7.2.3.	Comparación	60
7.3.	Resultados de planificación	60
7.3.1.	Qwen–CLIPS	60
7.3.2.	GPT directo	60
7.3.3.	Condición de control CLIPS	60
7.3.4.	Comparación	60

7.4.	Análisis de errores	60
7.4.1.	Errores de interpretación	60
7.4.2.	Errores de planificación	60
7.4.3.	Casos representativos	60
7.5.	Resultados de ejecución física	60
7.6.	Rendimiento computacional	60
7.6.1.	Latencia	60
7.6.2.	Recursos computacionales	60
7.6.3.	Tokens y coste	60
8.	Discusión	61
8.1.	Interpretación de los resultados respecto a la hipótesis	62
8.2.	Interpretación semántica	62
8.2.1.	Especialización de Qwen	62
8.2.2.	Capacidad de GPT	62
8.2.3.	Efecto de la variabilidad lingüística	62
8.3.	Planificación de acciones	62
8.3.1.	Expansión determinista mediante CLIPS	62
8.3.2.	Planificación generativa mediante GPT	62
8.3.3.	Separación entre errores de interpretación y planificación	62
8.4.	Comparación Qwen-CLIPS frente a GPT directo	62
8.4.1.	Exactitud	62
8.4.2.	Consistencia	62
8.4.3.	Latencia	62
8.4.4.	Recursos y coste	62
8.4.5.	Dependencia de infraestructura externa	62
8.5.	Generalización fuera de las plantillas GPSR	62
8.6.	Limitaciones y amenazas a la validez	62
8.6.1.	Dominio de tareas	62
8.6.2.	Dataset	62
8.6.3.	Modelo Qwen	62
8.6.4.	Modelo GPT	62
8.6.5.	Plataforma física	62
8.7.	Implicaciones para la planificación en robots de servicio	62

9. Conclusiones y trabajo futuro	63
9.1. Conclusiones principales	63
9.2. Cumplimiento de los objetivos	63
9.3. Evaluación de la hipótesis	63
9.4. Contribuciones de la investigación	63
9.4.1. Representación canónica de órdenes	63
9.4.2. Arquitectura Qwen–CLIPS	63
9.4.3. Comparación con planificación directa mediante GPT	63
9.4.4. Metodología para separar errores de interpretación y planificación	63
9.5. Trabajo futuro	63
10. Conclusiones y Trabajo Futuro	64
10.1. Conclusiones Principales	64
10.2. Propuestas de Trabajo Futuro	64

Índice de figuras

3.1. Arquitectura general ViRBot del robot Justina.	22
3.2. Arquitectura conceptual Qwen-CLIPS.	40
3.3. Arquitectura conceptual del planificador GPT directo.	43

Capítulo 1

Introducción

Los robots de servicio representan un campo de proyección para la inteligencia artificial y la automatización, con aplicaciones que van desde la asistencia a personas mayores hasta la gestión autónoma de entornos residenciales (1), donde deben interpretar instrucciones expresadas por personas y convertirlas en acciones que puedan ejecutarse mediante módulos de navegación, percepción, manipulación e interacción.(2).

La transformación de una instrucción en lenguaje natural a un plan robótico presenta dos problemas relacionados. El primero es semántico: identificar correctamente los objetivos, entidades, destinos y relaciones expresados por el usuario. El segundo es simbólico: convertir dicha interpretación en una secuencia de acciones pertenecientes al repertorio real del robot y con parámetros compatibles con el ejecutor.

Los sistemas basados en reglas, han demostrado ser robustos y predecibles en escenarios estructurados, como líneas de producción o laboratorios controlados (3). En particular, CLIPS utiliza hechos y reglas de producción para determinar qué transformaciones deben aplicarse a partir de un estado simbólico. Sin embargo, su rigidez los hace poco adaptables ante variaciones no previstas en el entorno o ante comandos expresados en lenguaje natural con alto grado de ambigüedad o complejidad (4).

Recientemente, los modelos de lenguaje grande (LLMs), como ChatGPT, han emergido como herramientas capaces de interpretar y generar lenguaje natural, e incluso de actuar como agentes de planificación autónomos (5). Su flexibilidad contextual permite traducir instrucciones verbales en secuencias de acciones, complementando así la solidez de los sistemas basados en reglas. No obstante, su naturaleza probabilística puede producir formatos inválidos, entidades inexistentes o acciones que no pertenecen a las capacidades del robot. Por lo tanto la falta de garantías formales de seguridad limitan

su uso directo en aplicaciones robóticas donde la integridad física y la predictibilidad son prioritarias (6).

Esta tesis estudia dos aproximaciones para resolver el problema. La primera es una arquitectura neuro-simbólica en la que un modelo Qwen2.5-7B, ajustado mediante QLoRA, transforma una orden en objetivos canónicos representados en JSON. Posteriormente, un módulo determinista normaliza y convierte esos objetivos en hechos, que son procesados por reglas de producción en CLIPS para generar el plan compatible con el ejecutor del robot. La segunda aproximación emplea un modelo GPT como planificador generativo directo. En este caso, el modelo recibe la orden, el contexto y el catálogo de acciones, produciendo una respuesta estructurada mediante JSON Schema que es revisada por un validador determinista antes de considerarse ejecutable.

Las dos aproximaciones no se combinan mediante un selector de planes durante la ejecución. Se implementan como arquitecturas independientes y se comparan experimentalmente bajo el mismo conjunto de órdenes, contexto y criterios de evaluación. Esta comparación permite analizar el compromiso entre interpretación semántica, consistencia, trazabilidad, flexibilidad, latencia, recursos computacionales y coste de uso.

1.1. Contexto y Motivación

El área de la robotica orientada al servicio doméstico ha evolucionado significativamente en la última década, impulsada por avances en percepción, planificación y interacción humano-robot (1).

El robot de servicio Justina, desarrollado en el Laboratorio de Bio-Robótica de la UNAM, integra módulos de percepción, navegación, manipulación e interacción mediante ROS 2. Para ejecutar una orden, el robot necesita una representación intermedia que conecte el lenguaje del usuario con las acciones ofrecidas por dichos módulos.

Estos sistemas se han diseñado para operar en entornos no estructurados como hogares, hospitales o centros de cuidado, donde deben realizar tareas que van desde la entrega de objetos hasta la asistencia en actividades de la vida diaria. En este contexto, la capacidad de planificar acciones de manera autónoma, segura y adaptativa se convierte en un requisito fundamental.

La planificación de acciones en robótica ha sido tradicionalmente abordada mediante sistemas basados en reglas (*Rule-Based Systems, RBS*), que ofrecen un marco predecible y verificable para la toma de decisiones (2). Sin embargo, una solución exclusivamente

simbólica puede generar planes consistentes cuando recibe objetivos correctamente estructurados, pero no resuelve directamente la interpretación del lenguaje natural. En contraste, un modelo de lenguaje puede interpretar distintas formulaciones de una orden, aunque su salida no es necesariamente compatible con las restricciones del robot. La motivación de este trabajo es evaluar si la separación entre interpretación neuronal y expansión simbólica permite obtener planes ejecutables y trazables, y contrastar sus resultados con los de un planificador basado exclusivamente en un modelo GPT.

El interés de la comparación no consiste en demostrar que un paradigma es universalmente superior. Se busca identificar que arquitectura presenta ventajas dependiendo del comando, así como que errores introduce cada etapa y cuáles son los costes asociados con su despliegue local o mediante una API externa.

1.2. Planteamiento del problema

Una orden puede contener variaciones lingüísticas, referencias espaciales, objetos, personas, destinos y varias subtarefas relacionadas. Para que el robot pueda ejecutarla, la orden debe convertirse en una secuencia válida de acciones primitivas, para esto el sistema basados en reglas CLIPS, fundamentado en la lógica simbólica y el ciclo *reconocer-actuar* (3), usa un conjunto de hechos y reglas bien definidos para producir una respuesta determinista y explicable, lo que lo hace idóneo para aplicaciones donde la seguridad es crítica, como entornos industriales o médicos.

La conversión por parte de CLIPS enfrenta los siguientes problemas:

1. **Rigidez interpretativa** No se pueden procesar comandos en lenguaje natural complejo o ambiguo.
2. **Falta de adaptabilidad:** Las reglas deben ser definidas a priori; cualquier situación no prevista en la base de conocimiento puede llevar al fracaso o a un comportamiento no deseado.
3. **Dificultad para gestionar la incertidumbre:** No manejan bien la información incompleta o cambiante del entorno en tiempo real.
4. **Escalabilidad limitada:** Mantener y extender un conjunto grande de reglas para cubrir todos los escenarios posibles resulta complejo y laborioso.

Por otro lado, los resultados deben satisfacer al menos las siguientes condiciones (6):

- Las entidades finales deben pertenecer al catálogo o ser tratadas explícitamente como desconocidas.

- Los objetivos deben conservar el significado y el orden causal de la instrucción.
- Las acciones generadas deben pertenecer al repertorio del ejecutor.
- Cada acción debe contener los parámetros requeridos.
- La secuencia debe respetar precondiciones simbólicas básicas, como la ubicación del robot o la posesión de un objeto.

Los modelos de lenguaje pueden ampliar la cobertura lingüística, aunque también pueden generar resultados inexistentes o inconsistentes. El problema de investigación se formula, por tanto, de la siguiente manera:

¿Qué diferencias de exactitud, consistencia, robustez y coste existen entre una arquitectura neuro-simbólica Qwen-CLIPS y un planificador generativo basado en GPT al convertir órdenes GPSR en planes para un robot de servicio?

1.3. Hipótesis

Para la interpretación y planeación de órdenes, una arquitectura neuro-simbólica compuesta por Qwen y CLIPS producirá una mayor proporción de planes estructuralmente válidos, trazables y consistentes que un planificador generativo basado directamente en GPT. Por otra parte, se espera que el planificador GPT directo presente mayor flexibilidad ante formulaciones lingüísticas novedosas, pero con mayor variabilidad, latencia y coste de inferencia mediante API.

Se espera que los sistemas:

- Mejore la tasa de éxito en la ejecución de tareas en entornos domésticos.
- Reduzca el tiempo de respuesta ante comandos complejos.
- Mantenga un comportamiento seguro y predecible en situaciones críticas.
- Permita una interacción más natural e intuitiva con usuarios no expertos.

La hipótesis se evaluará de forma separada para la interpretación de objetivos y para la generación del plan completo. De esta manera será posible determinar si un error proviene del modelo de lenguaje, de la conversión simbólica o de la cobertura de las reglas CLIPS.

1.4. Objetivos

Diseñar, implementar y evaluar una arquitectura neuro-simbólica Qwen–CLIPS para interpretar órdenes y generar planes compatibles con un robot de servicio, comparándola con un planificador generativo basado en GPT con salida estructurada.

■ Objetivos específicos

- Definir una representación canónica de objetivos y su correspondencia con las acciones disponibles en el ejecutor.
- Ajustar Qwen2.5-7B mediante QLoRA para convertir órdenes en objetivos canónicos representados en JSON.
- Implementar la normalización, validación y conversión de los objetivos generados por Qwen en hechos.
- Implementar reglas de producción en CLIPS que expandan objetivos de alto nivel en secuencias de acciones compatibles con el ejecutor.
- Diseñar un planificador GPT directo mediante un prompt, un JSON Schema y un validador determinista.
- Construir un conjunto experimental que incluya órdenes conocidas, paráfrasis, composiciones de objetivos, entradas ambiguas y casos fuera del dominio.
- Comparar las arquitecturas en términos de exactitud semántica, validez estructural, robustez, latencia, consumo de recursos, variabilidad y coste.
- Validar en simulación, robot físico y durante la RoboCup 2026 el resultado los planes generados.

■ Contribuciones esperadas

- Una representación intermedia de objetivos que conecta la interpretación lingüística con la planeación simbólica.
- Un modelo Qwen2.5-7B ajustado mediante QLoRA para traducir órdenes a objetivos canónicos.
- Una integración ROS 2–CLIPS que convierte dichos objetivos en planes compatibles con el ejecutor.
- Un planificador GPT directo restringido mediante prompt, JSON Schema y validación determinista.
- Un protocolo experimental reproducible para comparar planeación neuro-simbólica y planeación generativa.
- Un análisis de errores por etapa que diferencia fallos semánticos, estructurales, simbólicos y de ejecución.

1.5. Estructura del documento

Este documento está organizado en los siguientes capítulos que describen el sistema propuesto para el desarrollo de esta tesis, abarcando el planteamiento, desarrollo, implementación y validación del sistema híbrido de planificación. La estructura es la siguiente:

1. **Capítulo 1: Introducción.** Presenta el contexto, motivación, problemática, hipótesis y objetivos de la investigación, así como la justificación del enfoque híbrido en robótica de servicio doméstico.
2. **Capítulo 2: Antecedentes y estado del arte.** Revisa la evolución de los sistemas basados en reglas en robótica, las características de CLIPS, los enfoques modernos de aprendizaje automático y los sistemas híbridos existentes.
3. **Capítulo 3: Marco teórico y conceptual.** Describe los fundamentos de la planificación jerárquica, los sistemas de producción, los modelos de lenguaje grande (LLMs) y la arquitectura de integración propuesta.
4. **Capítulo 4: Metodología.** Detalla el diseño del sistema híbrido, incluyendo la definición de reglas en CLIPS, la integración con ChatGPT/Qwen, y los mecanismos de planificación dual y validación.
5. **Capítulo 5: Implementación.** Explica los entornos de desarrollo, la configuración técnica, la interfaz con APIs y las pruebas utilizados para validar la integración de los módulos.
6. **Capítulo 6: Escenarios de validación y experimentación.** Presenta el diseño experimental, los escenarios de prueba y las métricas cuantitativas y cualitativas utilizadas para evaluar el sistema.
7. **Capítulo 7: Análisis de resultados.** Compara el rendimiento del sistema híbrido frente al sistema basado solo en reglas, evalúa la efectividad de los LLMs y discute limitaciones y costos computacionales.
8. **Capítulo 8: Discusión.** Interpreta los resultados en relación con los objetivos, analiza ventajas y desventajas del enfoque híbrido, y explora implicaciones éticas y de seguridad.
9. **Capítulo 9: Contribuciones y relevancia.** Destaca la aportación técnica del marco híbrido, su aplicabilidad práctica en distintos sectores y su relevancia académica como benchmark.
10. **Capítulo 10: Conclusiones y trabajo futuro.** Resume los hallazgos principales y propone líneas de investigación futura, como el fine-tuning de LLMs y la

mejora de mecanismos de seguridad.

Capítulo 2

Antecedentes y Estado del Arte

Este capítulo presenta los trabajos y tecnologías relacionados con la transformación de instrucciones de lenguaje natural a planes para robots de servicio. La revisión se concentra en cuatro componentes relevantes para la tesis: sistemas basados en reglas, CLIPS, modelos de lenguaje para interpretación y planificación, y arquitecturas neuro-simbólicas. Los métodos de percepción, navegación o control se mencionan únicamente cuando contribuyen directamente a la interfaz entre la orden y el plan de tareas.

2.1. Sistemas Basados en Reglas

Los sistemas basados en reglas (*Rule-Based Systems, RBS*) constituyen uno de los paradigmas clásicos de la inteligencia artificial (3). Su utilización se consolidó con el desarrollo de los sistemas expertos durante las décadas de 1970 y 1980, en los que el conocimiento de especialistas humanos era representado explícitamente mediante hechos y reglas dentro de dominios bien delimitados (2). En robótica, la adopción de estos sistemas se popularizó en los años 80 y 90, particularmente en entornos industriales estructurados, donde la previsibilidad y seguridad eran prioritarias.

Este tipo de representación resulta útil cuando las condiciones bajo las cuales deben ejecutarse determinadas acciones pueden expresarse de manera explícita. Las reglas permiten relacionar estados, eventos u objetivos con decisiones específicas, facilitando la construcción de comportamientos reproducibles dentro de dominios previamente definidos. Entre las herramientas desarrolladas para implementar este paradigma se encuentra CLIPS (*C Language Integrated Production System*), desarrollado originalmente por la NASA. CLIPS proporciona mecanismos para representar conocimiento mediante

hechos y reglas de producción, junto con un motor de inferencia encargado de determinar qué reglas deben activarse de acuerdo con el estado de la base de conocimiento (7).

En cuanto a aplicaciones, los sistemas basados en reglas han demostrado su utilidad en diversos dominios robóticos:

1. **Robótica industrial:** Cadenas de montaje automatizadas, donde las tareas son repetitivas y el entorno está controlado. Los RBS se utilizan para secuenciar operaciones, gestionar excepciones y garantizar la seguridad de los operarios humanos (1).
2. **Robótica de servicio y doméstica:** En tareas simples como la entrega de objetos, navegación en interiores y asistencia a personas con movilidad reducida.
3. **Robótica espacial y de exploración:** Sistemas como Remote Agent de la NASA emplearon arquitecturas basadas en reglas para la planificación y ejecución autónoma de experimentos en misiones no tripuladas (8).

Su principal ventaja para esta tesis es que permiten establecer una correspondencia explícita entre un objetivo simbólico y las acciones emitidas para el robot. Sin embargo, la cobertura del sistema está limitada por las reglas y los tipos de hechos definidos previamente.

2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones

CLIPS es un lenguaje de programación basado en reglas, diseñado específicamente para la construcción de sistemas expertos y aplicaciones basadas en conocimiento (7). Concebido originalmente como una alternativa a los costosos sistemas implementados en (*LISP*, *LISt Processing*) en estaciones de trabajo especializadas, CLIPS destacó por su portabilidad, eficiencia y su capacidad para ejecutarse en hardware de bajo costo (3). Su adopción se extendió rápidamente en la comunidad de robótica e inteligencia artificial, convirtiéndose en una herramienta de referencia para la implementación de sistemas de producción (2).

2.2.1. Características Fundamentales de CLIPS

CLIPS es un entorno para el desarrollo de sistemas basados en conocimiento que ofrece hechos, plantillas, reglas de producción, funciones y un motor de inferencia que permite razonar sobre una base de conocimiento mediante reglas de producción (3).

En el contexto de esta tesis, las propiedades relevantes de CLIPS son:

1. **Encadenamiento hacia adelante (*forward chaining*):** CLIPS utiliza predominantemente encadenamiento hacia adelante. A partir de los hechos presentes en la base de conocimiento, el motor identifica las reglas cuyas condiciones se satisfacen y las incorpora a la agenda de ejecución. La ejecución de estas reglas puede modificar posteriormente la base de hechos y habilitar nuevas reglas (7).
2. **Representación explícita del conocimiento:** Además de reglas, CLIPS permite representar conocimiento mediante hechos ordenados y plantillas (*deftemplate*), así como mediante funciones definidas por el usuario en C (7). Esto facilita la integración con sistemas de percepción y control de bajo nivel (9).
3. **Trazabilidad de las reglas activadas:** Debido a que las decisiones dependen de hechos y reglas explícitas, es posible inspeccionar las reglas que son activadas durante una ejecución. Cuando se mantienen constantes los hechos iniciales, las reglas y la estrategia de resolución de conflictos, el comportamiento del motor puede reproducirse de manera determinista.
4. **Portabilidad:** CLIPS fue implementado en C con el propósito de facilitar su utilización en diferentes plataformas. Esta característica permite incorporar el motor de inferencia dentro de sistemas de software más amplios sin depender de una plataforma especializada (3,7).
5. **Integración con otros lenguajes:** CLIPS proporciona interfaces que permiten incorporarlo dentro de aplicaciones desarrolladas en otros lenguajes. En el caso de Python, esta comunicación puede realizarse mediante bindings externos que exponen las funciones del motor de CLIPS. Esto permite utilizar el sistema basado en reglas como un componente dentro de una arquitectura robótica más amplia.

Estas propiedades contribuyen a restringir y validar el comportamiento del sistema, pero no implican, por sí mismas, una garantía de seguridad ni constituyen un mecanismo de verificación formal.

CLIPS tampoco interpreta directamente lenguaje natural ni aprende reglas a partir de los ejemplos empleados en esta investigación. Para recibir una orden requiere una etapa externa que produzca objetivos estructurados. Además, la integración con ROS

2 no es una capacidad intrínseca de CLIPS, sino una interfaz implementada específicamente para comunicar el motor de reglas con los nodos del robot.

2.3. Modelos de lenguaje para interpretación de instrucciones robóticas

El desarrollo del aprendizaje automático, y en particular del aprendizaje profundo, ha ampliado las capacidades de los sistemas robóticos para procesar información compleja y entradas con un alto grado de variabilidad (10). A diferencia de los sistemas basados en reglas, en los que parte del conocimiento del dominio debe representarse explícitamente, los enfoques de aprendizaje automático permiten obtener representaciones y patrones a partir de datos (2).

En el procesamiento de información secuencial, arquitecturas como las redes neuronales recurrentes (RNN), LSTM y GRU permitieron modelar dependencias entre los elementos de una secuencia y fueron utilizadas en diferentes tareas relacionadas con el procesamiento del lenguaje natural (10).

Posteriormente, la arquitectura Transformer introdujo un enfoque basado principalmente en mecanismos de atención, permitiendo modelar relaciones entre los elementos de una secuencia sin depender exclusivamente de recurrencia (5). Esta arquitectura constituye la base de gran parte de los modelos de lenguaje de gran escala desarrollados posteriormente.

Los *Large Language Models* (LLMs) basados en Transformers pueden procesar y generar lenguaje natural utilizando el contexto de una instrucción para producir diferentes tipos de representaciones. En robótica, estos modelos se han utilizado para interpretar comandos complejos, traducir lenguaje natural a representaciones simbólicas e incluso generar planes a partir de descripciones de tareas de alto nivel (6).

Estas capacidades resultan relevantes para esta investigación debido a que permiten utilizar un modelo de lenguaje como interfaz entre las expresiones proporcionadas por el usuario y las representaciones estructuradas utilizadas posteriormente por los componentes de planificación.

2.3.1. Familia de modelos Qwen

Qwen es una familia de modelos de lenguaje desarrollada por el equipo Qwen de Alibaba Cloud. La familia incluye modelos de diferentes tamaños y variantes destinadas a tareas generales y al seguimiento de instrucciones (11). Estos modelos están basados en la arquitectura Transformer y pueden utilizarse en tareas como generación de texto, respuesta a preguntas, extracción de información y producción de contenido estructurado.

Qwen2.5 corresponde a una generación de esta familia que incluye modelos con diferentes números de parámetros. Para esta investigación se utiliza **Qwen/Qwen2.5-7B** como componente neuronal encargado de la interpretación de las instrucciones proporcionadas al robot. Los criterios que motivaron la selección de esta variante se desarrollan posteriormente.

En la arquitectura propuesta, Qwen2.5-7B funciona como intérprete semántico y no como generador del plan final. El modelo recibe una orden expresada en lenguaje natural y produce una secuencia de objetivos canónicos representada en formato JSON. Esta secuencia identifica los tipos de objetivo, las entidades, las ubicaciones y las relaciones extraídas de la instrucción. Posteriormente, un módulo determinista normaliza y valida la respuesta antes de convertirla en hechos que pueden ser procesados por CLIPS.

De esta manera, el modelo de lenguaje se utiliza específicamente para reducir la variabilidad lingüística presente en las instrucciones, mientras que la generación de las acciones necesarias para alcanzar los objetivos permanece bajo responsabilidad del componente simbólico.

Justificación de la selección de Qwen2.5-7B

La selección de Qwen2.5-7B como modelo base responde tanto a las características requeridas por la tarea como a las restricciones computacionales disponibles durante el desarrollo de esta investigación. La tarea planteada requiere interpretar instrucciones expresadas en lenguaje natural y transformarlas en una representación estructurada de objetivos canónicos, por lo que resulta conveniente utilizar un modelo con capacidad suficiente para seguir instrucciones, conservar las relaciones semánticas presentes en una orden y generar salidas con una estructura consistente.

Durante las etapas preliminares de desarrollo se realizaron pruebas con Qwen2.5-0.5B debido a sus menores requerimientos computacionales y a la posibilidad de efectuar rápidamente iteraciones de ajuste y evaluación. Estas pruebas tuvieron un carácter

exploratorio y permitieron validar inicialmente el flujo de entrenamiento y la representación utilizada para los objetivos.

Sin embargo, durante estas pruebas se observó que las respuestas del modelo presentaban una consistencia insuficiente para los requerimientos de la tarea, particularmente ante instrucciones con mayor complejidad o variabilidad lingüística. Debido a ello, se optó por utilizar una variante con mayor capacidad para los experimentos posteriores.

Estas observaciones iniciales se utilizaron como criterio de diseño para seleccionar la siguiente variante del modelo y no constituyen, por sí mismas, una comparación exhaustiva entre los diferentes tamaños de la familia Qwen2.5.

Se seleccionó finalmente Qwen2.5-7B como un compromiso entre capacidad del modelo y viabilidad computacional. Bajo la configuración de entrenamiento utilizada y los recursos computacionales disponibles durante esta investigación, la variante de siete mil millones de parámetros representó el mayor tamaño que resultó práctico para realizar de manera local los procesos de ajuste y experimentación. La plataforma utilizada estuvo compuesta por una GPU NVIDIA GeForce RTX 4070 Ti, un procesador Intel Core i7 de undécima generación y 32 GB de memoria RAM.

La utilización de modelos de mayor tamaño habría incrementado los requerimientos de memoria y procesamiento, además de reducir la flexibilidad para realizar múltiples iteraciones experimentales dentro de la infraestructura disponible. Por esta razón, se optó por mantener el modelo dentro de un tamaño que permitiera realizar tanto su adaptación como su evaluación localmente.

La elección de Qwen2.5-7B no pretende establecer que este modelo sea superior de manera general a otras familias de modelos de lenguaje. Existen diversas alternativas que podrían emplearse para una tarea de este tipo; sin embargo, la selección se realizó considerando el equilibrio entre capacidad para procesar instrucciones, disponibilidad del modelo, posibilidad de ejecutarlo y adaptarlo localmente, y los recursos computacionales disponibles. De esta manera, Qwen2.5-7B constituye una solución viable para evaluar la integración de un modelo de lenguaje especializado dentro de la arquitectura neuro-simbólica propuesta.

2.3.2. Modelos GPT

Los modelos GPT constituyen otra familia de modelos generativos basados en la arquitectura Transformer. En esta tesis, el modelo seleccionado se utiliza mediante la API de OpenAI, en lugar de desplegar sus parámetros dentro de la infraestructura local.

La API recibe la instrucción y el contexto proporcionado para la tarea y devuelve una respuesta generada por el modelo.

En el capítulo de implementación se especifican el modelo y la configuración utilizados durante los experimentos, debido a que una denominación general como “ChatGPT” no identifica de manera reproducible un modelo específico accesible mediante API (12).

En la arquitectura experimental GPT directo, el modelo recibe la orden, el catálogo de entidades, el repertorio de acciones, las restricciones del dominio y ejemplos de respuesta. A diferencia de Qwen implementado en la arquitectura neuro-simbólica, este modelo genera tanto la representación de los objetivos como la secuencia propuesta de acciones. Por ello, constituye un planificador generativo independiente y no un módulo interno de la cadena Qwen-CLIPS.

La respuesta del modelo se solicita mediante una salida estructurada definida a partir de JSON Schema. La documentación de la API denomina *Structured Outputs* al mecanismo mediante el cual la respuesta generada debe ajustarse al esquema proporcionado (12). Este mecanismo facilita la integración de la salida con los componentes posteriores del sistema, aunque el cumplimiento del esquema no garantiza por sí mismo la validez semántica del contenido generado.

Los distintos niveles de validación empleados para las salidas de los modelos se describen posteriormente en la subsección de salidas estructuradas y validación.

El uso de una API externa introduce variables que deben incluirse en la evaluación, como latencia de comunicación, disponibilidad del servicio, tokens de entrada y salida, coste monetario y posibles cambios entre versiones del modelo. En consecuencia, las pruebas deben fijar y documentar el modelo, los parámetros de generación y el prompt.

2.3.3. Interpretación semántica y generación de planes robóticos

Para esta tesis conviene distinguir dos funciones:

Interpretación semántica: Transformación de una orden en una representación intermedia de objetivos, entidades y relaciones.

Planificación generativa: Producción directa de una secuencia de acciones a partir de la orden, el contexto y las capacidades declaradas del robot.

En la arquitectura Qwen-CLIPS estas funciones se mantienen separadas: Qwen realiza la interpretación semántica de la orden y CLIPS expande los objetivos obtenidos

en una secuencia de acciones. En la arquitectura GPT directo, ambas funciones se concentran en un único modelo, que produce tanto la representación de los objetivos como el plan propuesto. Esta distinción permite comparar el efecto de mantener una etapa simbólica explícita frente a delegar conjuntamente la interpretación y la composición del plan a un modelo generativo.

La interpretación semántica puede evaluarse comparando la secuencia de objetivos producida por el modelo con una anotación de referencia. La planificación requiere criterios adicionales: pertenencia de las acciones al repertorio del robot, validez de parámetros, conservación de dependencias, cumplimiento de precondiciones y compatibilidad con el ejecutor. Evaluar ambas etapas de forma separada permite localizar el origen de los errores.

2.3.4. Limitaciones de los modelos de lenguaje en robótica

El uso de un modelo de lenguaje introduce riesgos específicos: generación de entidades o acciones inexistentes, incumplimiento del formato, variabilidad entre ejecuciones, sensibilidad al *prompt* y costes de inferencia. Asimismo, una instrucción ambigua puede dar lugar a una interpretación plausible pero diferente de la intención del usuario.

Estas limitaciones son especialmente relevantes cuando la salida se utiliza para controlar un sistema físico. Por ello, una respuesta lingüísticamente coherente no debe considerarse automáticamente un plan ejecutable. En esta investigación se separan la generación, la validación estructural y la validación semántica; los módulos de ejecución del robot conservan la responsabilidad sobre restricciones físicas y condiciones observadas durante la tarea.

2.3.5. Salidas estructuradas y validación

Una estrategia para integrar modelos de lenguaje con software convencional consiste en restringir la salida a una estructura legible por máquina. JSON permite representar objetivos, parámetros y acciones, mientras que JSON Schema permite comprobar tipos, campos obligatorios, enumeraciones y restricciones estructurales.

No obstante, una respuesta conforme al esquema puede seguir siendo semánticamente incorrecta. Por ejemplo, puede incluir una ubicación que no pertenece al mapa o aplicar una acción válida al tipo de entidad equivocado. En consecuencia, la validación debe separar al menos tres niveles:

1. Sintaxis JSON.
2. Conformidad con el esquema.
3. Validez semántica respecto a catálogos, acciones y precondiciones.

Esta separación es aplicable tanto a los objetivos producidos por Qwen como al plan generado directamente por GPT.

2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA

El ajuste completo de un modelo de lenguaje requiere actualizar todos sus parámetros y mantener estados adicionales durante el entrenamiento. Los métodos de adaptación eficiente en parámetros permiten reducir este coste al entrenar únicamente un subconjunto reducido de parámetros, manteniendo congelada gran parte del modelo preentrenado (13, 14). LoRA (*Low-Rank Adaptation*) introduce matrices entrenables de bajo rango en determinadas capas del modelo, manteniendo congelados los pesos originales (14), mientras que QLoRA combina estos adaptadores con la cuantización del modelo base para reducir significativamente el uso de memoria durante el ajuste fino (15).

En esta investigación se utiliza QLoRA para especializar Qwen2.5-7B en la conversión de órdenes a objetivos canónicos. Los detalles del conjunto de datos, cuantización, hiperparámetros y plataforma pertenecen al capítulo de implementación.

2.4. Arquitecturas neuro-simbólicas

Las arquitecturas neuro-simbólicas buscan combinar las capacidades de aprendizaje y generalización de los modelos neuronales con mecanismos explícitos de representación de conocimiento y razonamiento simbólico. Mientras que los modelos neuronales son especialmente adecuados para procesar información no estructurada y aprender representaciones a partir de datos, los sistemas simbólicos permiten expresar conocimiento mediante estructuras discretas, reglas y relaciones que pueden ser utilizadas durante un proceso de inferencia (16, 17). El objetivo de este enfoque es aprovechar las fortalezas de ambos paradigmas en lugar de utilizar cada uno de manera aislada.

La integración entre ambos componentes puede realizarse de diferentes maneras. Un modelo neuronal puede, por ejemplo, transformar una entrada no estructurada en una representación simbólica que posteriormente es procesada por un sistema de razonamiento. En la dirección opuesta, el conocimiento simbólico puede utilizarse para

imponer restricciones, validar resultados o proporcionar información adicional durante el aprendizaje o la inferencia del modelo neuronal (17). Por tanto, una arquitectura neuro-simbólica no implica necesariamente que el aprendizaje neuronal y el razonamiento simbólico se realicen dentro de un único modelo.

Esta separación resulta particularmente útil cuando la entrada del sistema se encuentra expresada en lenguaje natural, pero la ejecución posterior requiere representaciones estructuradas y operaciones sujetas a reglas explícitas. En este tipo de arquitecturas, el modelo neuronal puede encargarse de interpretar la variabilidad lingüística y transformarla en una representación formal o canónica, mientras que un componente simbólico utiliza dicha representación para realizar razonamiento o planificación.

Un ejemplo de integración entre modelos de lenguaje y planificación simbólica es la arquitectura LLM+P, propuesta en *Empowering Large Language Models with Optimal Planning Proficiency* (18). En este enfoque, el modelo de lenguaje no se utiliza directamente como el mecanismo encargado de buscar la secuencia final de acciones. En su lugar, participa en la transformación de una descripción expresada en lenguaje natural hacia una representación formal que posteriormente puede ser procesada por un planificador clásico.

Para esta representación se utiliza PDDL (*Planning Domain Definition Language*), un lenguaje destinado a describir problemas de planificación automática. De manera general, PDDL permite representar los elementos que caracterizan al dominio de planificación, como las acciones disponibles, sus parámetros, precondiciones y efectos, así como los elementos específicos de un problema, entre ellos los objetos involucrados, el estado inicial y el objetivo que debe alcanzarse.

Una vez obtenida la representación formal del problema, la búsqueda de una secuencia de acciones que satisfaga el objetivo se delega a un planificador clásico. De esta manera, el modelo de lenguaje participa principalmente en la interpretación y formalización de la descripción proporcionada, mientras que el razonamiento necesario para obtener el plan se realiza mediante un mecanismo simbólico.

De manera simplificada, este flujo puede representarse como:

Lenguaje natural $\rightarrow$ Modelo de lenguaje $\rightarrow$ PDDL $\rightarrow$ Planificador clásico $\rightarrow$ Plan.

Esta separación permite asignar a cada componente una función distinta: el modelo de lenguaje se encarga principalmente de interpretar expresiones lingüísticas y convertirlas

en una representación estructurada, mientras que el planificador trabaja sobre dicha representación para determinar una secuencia de acciones que satisfaga las restricciones del problema (18). Este enfoque constituye un ejemplo de cómo un componente neuronal puede utilizarse como interfaz entre lenguaje natural y un mecanismo simbólico de planificación.

La arquitectura principal de esta tesis sigue una idea conceptualmente relacionada, aunque no utiliza PDDL ni reproduce directamente la arquitectura LLM+P. En este trabajo, el modelo Qwen2.5 (19), adaptado al dominio de tareas de servicio del robot, recibe una instrucción expresada en lenguaje natural y la transforma en una representación canónica de los objetivos solicitados. Esta representación constituye la interfaz entre el componente neuronal y el componente simbólico de la arquitectura.

El propósito de esta primera etapa es abstraer parte de la variabilidad presente en el lenguaje natural, como diferentes estructuras gramaticales, sinónimos o formas alternativas de expresar una misma intención. De este modo, distintas instrucciones que representan una intención equivalente pueden transformarse en objetivos con una estructura común antes de ser procesadas por el sistema de planificación.

En la segunda etapa, los objetivos obtenidos son enviados a CLIPS. A diferencia del modelo neuronal, CLIPS no interpreta directamente la instrucción original en lenguaje natural. Su función consiste en trabajar sobre los símbolos previamente obtenidos, incorporar los objetivos al estado de conocimiento y aplicar las reglas definidas para descomponerlos y expandirlos en las acciones necesarias para su ejecución. De esta manera, la interpretación lingüística y la expansión simbólica del plan permanecen como responsabilidades diferenciadas dentro de la arquitectura.

Por tanto, aunque LLM+P y la arquitectura propuesta en esta tesis utilizan mecanismos y representaciones diferentes, ambas ilustran una separación entre interpretación neuronal y razonamiento simbólico. En LLM+P, el modelo de lenguaje genera una representación PDDL que posteriormente es procesada por un planificador clásico; en esta tesis, Qwen genera objetivos canónicos que son posteriormente procesados mediante reglas de producción en CLIPS. La semejanza se encuentra, por tanto, en la separación funcional entre ambos paradigmas y no en el formalismo simbólico empleado.

Esta división permite que el modelo neuronal se concentre en el problema de correspondencia entre lenguaje natural y representaciones canónicas, mientras que el conocimiento sobre la estructura de las tareas, sus relaciones y su descomposición se mantiene explícitamente en la base de reglas de CLIPS. En consecuencia, una modificación en las

reglas de planificación no requiere necesariamente modificar el modelo de lenguaje y, de forma análoga, las mejoras realizadas sobre el componente de interpretación pueden introducirse sin sustituir el mecanismo simbólico de planificación.

En esta tesis, por tanto, la integración neuro-simbólica se describe como una composición secuencial: Qwen transforma el lenguaje natural en objetivos canónicos y CLIPS expande dichos objetivos mediante reglas de producción. Los componentes no actúan como dos planificadores concurrentes ni se realiza una fusión automática de planes generados independientemente. El denominado planificador GPT directo se implementa como una arquitectura alternativa con fines de comparación experimental y no como un componente adicional de la cadena neuro-simbólica Qwen-CLIPS.

2.5. Síntesis y conexión con la propuesta

Los antecedentes revisados muestran que los sistemas simbólicos permiten mantener una representación explícita del conocimiento utilizado durante la planificación, mientras que los modelos de lenguaje ofrecen mecanismos para procesar la variabilidad presente en las instrucciones expresadas en lenguaje natural. En una arquitectura híbrida, la combinación de ambos enfoques permite separar el problema de interpretación lingüística del proceso encargado de expandir los objetivos en acciones ejecutables.

Esta tesis estudia dicha separación mediante una comparación controlada entre una arquitectura secuencial Qwen-CLIPS y un planificador GPT directo. Adicionalmente, se utiliza una condición experimental de control en la que los objetivos de referencia son proporcionados directamente a CLIPS. Esta condición permite observar el comportamiento de la etapa simbólica cuando los objetivos de entrada corresponden directamente a la referencia, evitando introducir los errores provenientes de la etapa de interpretación del lenguaje natural. De esta manera, es posible distinguir los errores asociados con la interpretación lingüística de aquellos relacionados con la cobertura y expansión de las reglas de planificación. Esta condición no constituye una tercera arquitectura de planificación, sino un mecanismo auxiliar para aislar los errores correspondientes a la etapa simbólica.

El modelo Qwen2.5-7B ajustado mediante QLoRA recibe una instrucción y produce una secuencia de objetivos canónicos representada en JSON. Un módulo determinista valida y normaliza la respuesta, la convierte en hechos y la envía al motor CLIPS. Finalmente, las reglas de producción expanden cada objetivo en acciones compatibles

con el ejecutor del robot.

Como segunda arquitectura evaluada, se implementa un planificador basado en GPT que genera conjuntamente una representación estructurada de los objetivos interpretados y del plan propuesto. Su salida se restringe mediante JSON Schema y posteriormente se somete a validación determinista. Ambas arquitecturas se evalúan de forma independiente; por tanto, el trabajo no propone una selección concurrente ni una fusión automática de los planes generados.

La contribución experimental consiste en comparar ambos enfoques bajo el mismo conjunto de órdenes, catálogos y acciones, separando los errores de interpretación de los errores de planificación y cuantificando su desempeño mediante métricas de exactitud, ejecutabilidad, consistencia, latencia, uso de recursos y coste.

Capítulo 3

Formalización del problema y arquitectura conceptual

El problema abordado en esta tesis consiste en transformar una instrucción expresada en lenguaje natural en una secuencia de acciones que pueda ser procesada por el sistema de ejecución de un robot de servicio. Esta transformación requiere relacionar dos niveles de representación diferentes: por una parte, la intención expresada por el usuario y, por otra, las acciones concretas disponibles en el robot.

Como se revisó en el capítulo anterior, los modelos de lenguaje proporcionan mecanismos para procesar la variabilidad presente en las instrucciones lingüísticas, mientras que los sistemas simbólicos permiten representar conocimiento y relaciones mediante estructuras explícitas. A partir de estos antecedentes, en este capítulo se formaliza el problema particular considerado en la investigación y se establecen las representaciones utilizadas para describir órdenes, objetivos y planes.

El dominio experimental toma como referencia las tareas de General Purpose Service Robot (GPSR) de RoboCup@Home. Sin embargo, GPSR se utiliza como una fuente controlada de tareas, entidades y escenarios, y no como una restricción que obligue al usuario a reproducir literalmente las formulaciones lingüísticas utilizadas en la competencia.

Finalmente, se presentan las dos arquitecturas que serán evaluadas. La primera utiliza Qwen2.5-7B como intérprete semántico y CLIPS como componente encargado de expandir los objetivos obtenidos en acciones ejecutables. La segunda delega tanto la interpretación como la generación del plan a un modelo GPT. Ambas arquitecturas utilizan el mismo dominio conceptual, pero distribuyen de manera diferente la respon-

sabilidad de transformar una orden en un plan.

3.1. Contexto arquitectónico y alcance del sistema

3.1.1. Integración dentro de la arquitectura ViRBot

La investigación se desarrolla sobre el robot de servicio Justina y su arquitectura de software ViRBot(ver Figura 3.1). Esta arquitectura integra diferentes capacidades necesarias para la operación del robot, entre ellas percepción, interacción humano-robot, navegación, manipulación, representación de conocimiento, planificación y ejecución.

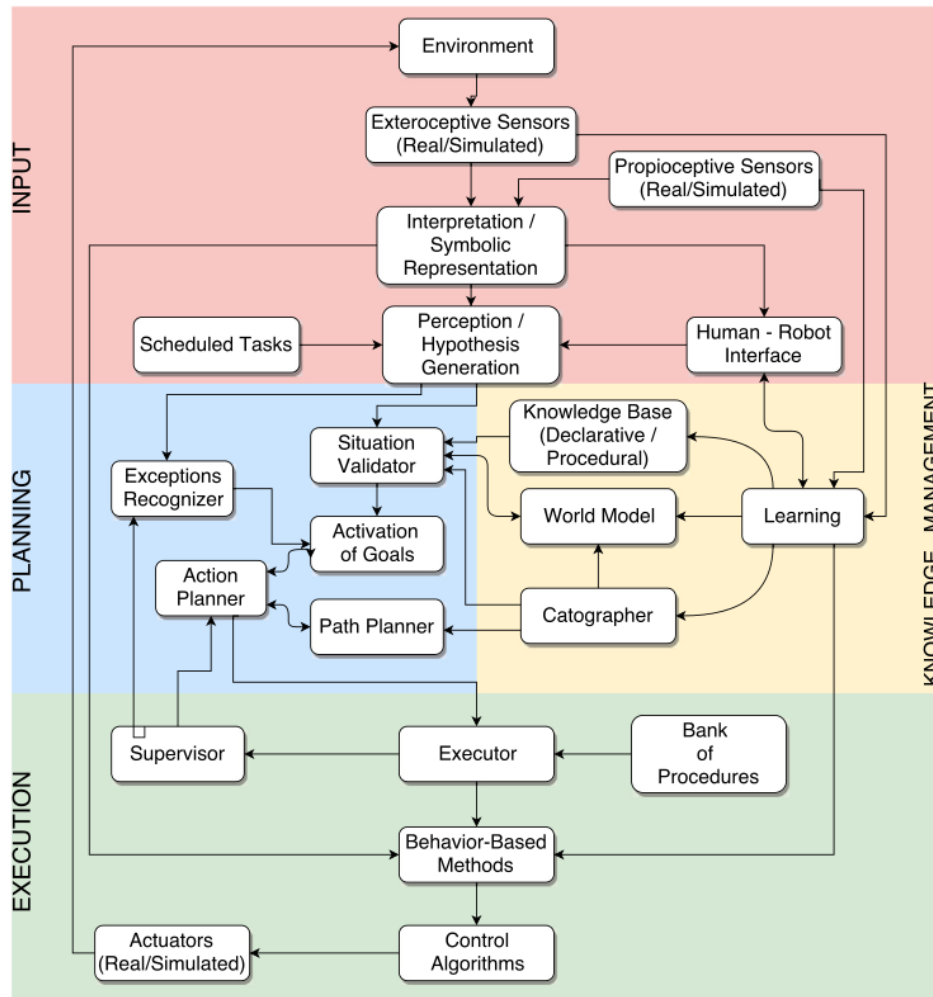


Figura 3.1: Arquitectura general ViRBot del robot Justina.

El sistema desarrollado en esta tesis se ubica entre la recepción de una instrucción

expresada en lenguaje natural y los componentes encargados de ejecutar las acciones correspondientes. Por tanto, no pretende sustituir los módulos de navegación, percepción o manipulación del robot, sino proporcionar una representación ordenada de las acciones que dichos módulos deben realizar.

De manera simplificada, el flujo general puede expresarse como:

$$\boxed{\text{orden en lenguaje natural} \rightarrow \text{interpretación y planificación} \rightarrow \text{plan} \rightarrow \text{ejecución}} \quad (3.1)$$

La contribución estudiada se concentra principalmente en las transformaciones anteriores a la ejecución física. Los módulos existentes de Justina son los responsables de intentar realizar las acciones especificadas en el plan.

3.1.2. Alcance del problema abordado

El trabajo se concentra en la transformación de instrucciones en lenguaje natural en planes de acciones compatibles con las capacidades del robot. Una instrucción puede contener varias subtareas relacionadas que deben ser identificadas antes de generar las acciones correspondientes compatibles con sus capacidades. Los módulos de percepción, navegación y manipulación se consideran externos al problema de planificación estudiado y se utilizan posteriormente para realizar las acciones producidas.

3.2. GPSR como dominio de referencia

3.2.1. La prueba General Purpose Service Robot

General Purpose Service Robot (GPSR) es una prueba de RoboCup@Home orientada a evaluar la capacidad de un robot de servicio para comprender instrucciones expresadas en lenguaje natural y ejecutar tareas que pueden requerir la combinación de distintas habilidades. Entre ellas se encuentran la navegación, la búsqueda y reconocimiento de personas u objetos, la manipulación y la interacción humano-robot(?).

Una característica importante de GPSR es que una instrucción puede representar una tarea compuesta. Por ejemplo, una orden puede requerir que el robot se desplace hacia una habitación, localice un objeto, lo tome y posteriormente lo entregue a una persona. En consecuencia, el problema no consiste únicamente en reconocer pala-

bras o entidades aisladas, sino en identificar las diferentes intenciones contenidas en la instrucción y conservar las relaciones existentes entre ellas.

En la prueba considerada en esta investigación se proporcionan tres comandos al robot. Al comenzar, el robot se desplaza hacia un *Instruction Point*, que funciona como el punto de interacción con un operador. Desde esta posición recibe una instrucción y, una vez que ha realizado la tarea solicitada, regresa al mismo punto para recibir el siguiente comando. La determinación de que una tarea ha sido completada de manera suficiente para continuar con la prueba corresponde a un árbitro.

La entrega de las órdenes se realiza de manera secuencial. Por tanto, el robot no recibe un nuevo comando mientras la ejecución de la tarea anterior continúe en curso. Una vez concluida la tarea y después de regresar al *Instruction Point*, puede solicitar la siguiente instrucción hasta completar los tres comandos contemplados por la prueba.

Cuando el robot no logra comprender adecuadamente la instrucción recibida mediante su sistema de entrada de audio, puede solicitar explícitamente que la orden sea repetida. En este caso, el operador expresara la misma tarea mediante una construcción lingüística más sencilla, pero manteniendo los objetivos contenidos en la instrucción original. La reformulación no constituye una modificación de la tarea ni permite que el operador indique directamente al robot cómo resolverla.

Este mecanismo es particularmente relevante para la presente investigación, pues permite distinguir entre la tarea que debe realizarse y la forma lingüística utilizada para comunicarla. Dos instrucciones con estructuras sintácticas diferentes pueden representar los mismos objetivos y, por tanto, deberían conducir a una interpretación semántica equivalente.

Durante la ejecución no se contempla que una persona intervenga para descomponer la orden, seleccionar las acciones del robot o corregir manualmente el plan generado. La interpretación de la instrucción y la determinación de las acciones necesarias corresponden al propio sistema robótico. De esta manera, GPSR proporciona un escenario apropiado para evaluar la transformación de instrucciones en lenguaje natural en representaciones semánticas y planes de acción.

3.2.2. Generación de comandos y conocimiento del dominio

Una característica relevante de GPSR es que los comandos utilizados durante la prueba no se seleccionan únicamente a partir de una lista fija de oraciones. El reglamento establece que las tareas se construyen mediante el *CommandGenerator* oficial

de RoboCup@Home ([?],[?]). Sobre esta herramienta se realizaron posteriormente adaptaciones específicas para construir el conjunto de datos empleado en el entrenamiento y evaluación del modelo. Dichas modificaciones se describen en el capítulo de metodología.

El generador define diferentes familias de instrucciones relacionadas con personas y objetos. Entre ellas se encuentran tareas para desplazarse hacia una ubicación, encontrar personas u objetos, contar entidades, obtener información, seguir o guiar personas, tomar objetos y entregarlos o colocarlos en una ubicación determinada. Algunas de estas estructuras permiten incorporar continuaciones que producen comandos compuestos. De esta manera, una instrucción inicial puede extenderse con subtareas posteriores que dependen del resultado de las anteriores ([?]).

De forma simplificada, el proceso puede representarse como:

$$\boxed{\text{familia de tarea} + \text{estructura lingüística} + \text{entidades del dominio} \rightarrow \text{comando GPSR}} \quad (3.2)$$

Las estructuras del generador utilizan elementos variables para representar entidades y relaciones del dominio. Por ejemplo, una plantilla puede contener una habitación, una persona, un objeto o una ubicación que posteriormente son sustituidos por valores concretos. De forma similar, diferentes verbos pueden representar una intención relacionada; por ejemplo, una tarea de navegación puede expresarse mediante verbos como *go* o *navigate*, mientras que una búsqueda puede utilizar expresiones como *find*, *locate* o *look for* ([?]).

El conocimiento utilizado para realizar estas sustituciones se mantiene separado de las estructuras de los comandos. El generador distingue, entre otros elementos, nombres de personas, habitaciones, ubicaciones, superficies de colocación, objetos y categorías de objetos ([?]). La herramienta permite recibir este conocimiento desde un directorio externo y señala el repositorio *CompetitionTemplate* como referencia para su organización ([?],[?]).

En *CompetitionTemplate*, la información específica de una competencia se organiza en archivos relacionados con el mapa y las ubicaciones, los nombres de personas y los objetos disponibles ([?]). Esta separación resulta importante porque permite mantener las estructuras generales de las tareas mientras se modifican las entidades concretas presentes en una edición o escenario determinado.

Conceptualmente, puede representarse el dominio de referencia como:

$$D_{\text{GPSR}} = (T, K), \quad (3.3)$$

donde T representa las familias de tareas contempladas por el generador y K el conocimiento específico utilizado para instanciarlas. La estructura detallada de K y su utilización en esta investigación se formalizan en la Sección 3.3.3.

3.2.3. Dominio de referencia y alcance lingüístico

La utilización de GPSR como referencia no implica que el sistema desarrollado en esta tesis deba reconocer únicamente las frases producidas literalmente por el generador oficial. Esta distinción resulta especialmente importante debido a que una misma intención puede expresarse mediante diferentes construcciones lingüísticas.

El propio procedimiento de GPSR 2026 introduce variación sobre las órdenes generadas. Una vez producido un comando mediante el generador oficial, el reglamento establece que éste se proporciona a un modelo de lenguaje para obtener una formulación similar. Además, una instrucción puede ser reformulada hasta tres veces, utilizando expresiones progresivamente más simples cuando sea necesario (?).

Por ejemplo, el reglamento presenta una transformación equivalente para pasar de una instrucción como:

“Get me a coke from the kitchen.”

a una formulación más explícita, como:

“Go to the kitchen, find a coke, and bring it to me.”

Aunque la estructura superficial de ambas expresiones es diferente, la tarea subyacente permanece esencialmente sin cambios. Esto permite distinguir dos niveles: forma lingüística e intención semántica.

Esta distinción es central para la presente investigación. GPSR proporciona un conjunto controlado de tareas y entidades, pero la entrada lingüística puede presentar diferentes formas de expresar una misma intención. El objetivo del intérprete no consiste, por tanto, en memorizar las plantillas del generador, sino en establecer una correspondencia entre distintas realizaciones lingüísticas y una representación semántica común.

Usando los ejemplos anteriores, ambos deben corresponder a la misma secuencia de objetivos:

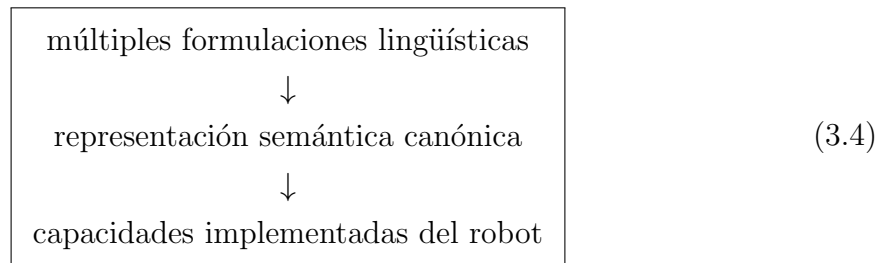
```

go(kitchen)
find(coke, kind=object)
take(coke)
deliver(coke, to=me)

```

Por tanto, en esta tesis GPSR se utiliza como una fuente controlada para definir las tareas, entidades y relaciones que forman parte del dominio experimental, pero no como una gramática que limite las únicas formulaciones aceptables por el sistema.

Puede resumirse esta relación mediante:



En consecuencia, la entrada lingüística puede variar en su formulación, pero la interpretación permanece acotada por el dominio semántico y las capacidades operativas disponibles. Una instrucción puede ser perfectamente comprensible para una persona y, aun así, quedar fuera del alcance del sistema si requiere una intención que no puede representarse mediante los objetivos contemplados o una capacidad que el robot no posee.

El dominio experimental de esta investigación considera principalmente instrucciones en inglés, en correspondencia con las estructuras utilizadas por el generador y con el escenario de evaluación. Por ello, los resultados relacionados con variabilidad lingüística deben interpretarse como capacidad para procesar diferentes formulaciones de las tareas soportadas dentro de dicho idioma, y no como evidencia de independencia respecto al idioma o de comprensión irrestricta del lenguaje natural.

3.3. Formalización de órdenes, objetivos y planes

La transformación de una instrucción en lenguaje natural en un plan para un robot de servicio implica relacionar representaciones con diferentes niveles de abstracción. Por una parte, el usuario expresa una tarea mediante lenguaje natural; por otra, el sistema de ejecución requiere una secuencia de acciones compatibles con las capacidades

disponibles del robot.

En esta investigación se introduce una representación semántica intermedia que permite separar ambos niveles. De manera general, el problema se representa mediante:

$$\boxed{x \rightarrow G \rightarrow P} \quad (3.5)$$

donde x representa la instrucción en lenguaje natural, G una secuencia ordenada de objetivos semánticos y P el plan compuesto por acciones ejecutables.

La primera transformación, $x \rightarrow G$, corresponde a la interpretación de la orden y busca determinar qué solicita el usuario. La segunda, $G \rightarrow P$, corresponde a la planificación y determina qué acciones son necesarias para atender los objetivos identificados.

Esta separación constituye la base conceptual utilizada posteriormente para comparar las arquitecturas Qwen-CLIPS y GPT directo.

3.3.1. Orden en lenguaje natural

Se denota mediante x una instrucción expresada en lenguaje natural dentro del dominio considerado en esta investigación.

Una instrucción puede representar una tarea simple o una composición de varias subtareas relacionadas. Por ejemplo:

“Go to the kitchen, find an apple and bring it to me.”

incluye navegación, búsqueda, manipulación y entrega de un objeto. Por tanto, una instrucción lingüística no corresponde necesariamente con una única acción del robot.

Incluso una expresión breve puede implicar varias operaciones. Considérese:

“Bring me an apple from the kitchen.”

Dentro del dominio utilizado en esta investigación, esta orden puede representarse mediante la siguiente secuencia semántica:

```
go(kitchen)
find(apple, kind=object)
take(apple)
deliver(apple, to=me)
```


La interpretación no consiste, por tanto, únicamente en identificar palabras clave. También debe reconocer las subtareas explícitas o implícitas, las entidades involucradas y las relaciones necesarias entre ellas.

El resultado de este proceso se representa mediante una secuencia:

$$G = (g_1, g_2, \dots, g_n),$$

donde cada g_i corresponde a un objetivo semántico. El número de objetivos depende de la tarea representada por la instrucción y no necesariamente de su longitud lingüística.

3.3.2. Equivalencia semántica entre formulaciones

Como se describió en la Sección 3.2, una misma tarea puede expresarse mediante diferentes formulaciones lingüísticas. Esta propiedad resulta especialmente relevante en GPSR, donde una orden puede ser repetida o reformulada sin modificar los objetivos que debe cumplir el robot.

En esta investigación se consideran semánticamente equivalentes dos instrucciones cuando, dentro del dominio utilizado, representan la misma secuencia de objetivos.

Esta relación se denota mediante $x_1 \sim x_2$. Idealmente:

$$\boxed{x_1 \sim x_2 \quad \Rightarrow \quad G_1 = G_2} \tag{3.6}$$

donde G_1 y G_2 son las representaciones semánticas obtenidas para cada instrucción. Por ejemplo:

“Fetch a bottle from the living room and throw it in the trash.”

y:

“Go to the living room, grab a bottle, and toss it in the trash.”

presentan estructuras lingüísticas diferentes, pero pueden corresponder a la misma representación:

```
go(living room)
find(bottle, kind=object)
take(bottle)
place(tomato soup, in=trash)
```

Esta propiedad se utiliza en la presente investigación para representar de forma común órdenes lingüísticamente diferentes que conservan la misma intención o representación semántica canónica..

3.3.3. Entidades y conocimiento del dominio

La interpretación de una instrucción requiere información acerca de las entidades que pueden participar en las tareas consideradas.

En esta investigación, el conocimiento relevante del dominio se representa mediante:

$$K = \{K_P, K_O, K_C, K_L, K_R\}, \quad (3.7)$$

donde:

- K_P representa las personas o nombres conocidos;
- K_O representa los objetos;
- K_C representa las categorías de objetos;
- K_L representa las ubicaciones específicas;
- K_R representa las habitaciones.

Esta información permite determinar el tipo de las entidades utilizadas en una instrucción. Por ejemplo, si **Robin** pertenece al conjunto de personas y **apple** al conjunto de objetos, pueden utilizarse representaciones como:

```
find(Robin, kind=person)
find(apple, kind=object)
```

Ambos ejemplos utilizan el mismo tipo general de objetivo, pero operan sobre entidades semánticamente diferentes.

El conocimiento K cumple dos funciones principales. En primer lugar, permite verificar que las entidades nominales utilizadas pertenezcan al dominio. Una salida que introduzca arbitrariamente una persona, objeto o ubicación no debe considerarse correcta únicamente porque su estructura sea válida.

En segundo lugar, permite verificar la compatibilidad entre las entidades y los objetivos en los que participan.

Las referencias presentes en una instrucción pueden adoptar diferentes formas. En esta investigación se distinguen tres casos principales:

1. **Entidades nominales:** referencias explícitas a una entidad conocida, como **Robin**, **apple** o **kitchen**.

2. **Entidades genéricas:** referencias a una clase de entidad, como `person`, `drink` o `object`.
3. **Entidades descritas mediante propiedades:** referencias que requieren identificar una entidad a partir de alguna característica, como una persona saludando, una persona sentada o el objeto de mayor tamaño.

Por ejemplo:

```
find(person, kind=person, gesture='waving person')
```

no identifica inicialmente a una persona mediante un nombre, sino mediante una característica observable.

De manera similar:

```
find(drink, kind=object, property=largest)
```

representa la búsqueda de un objeto perteneciente a una categoría y sujeto a una propiedad adicional.

Estas distinciones permiten conservar la información necesaria para que las etapas posteriores puedan determinar cómo realizar la tarea.

3.3.4. Representación canónica de objetivos

El resultado de la interpretación se expresa mediante una secuencia ordenada de objetivos:

$$G = (g_1, g_2, \dots, g_n).$$

Cada objetivo g_i contiene un tipo de intención y los argumentos necesarios para describirla. Por ejemplo:

```
go(kitchen)
find(Robin, kind=person)
deliver(apple, to=Robin)
```

Se denota mediante $\mathcal{G}$ el vocabulario de tipos de objetivos semánticos utilizados en esta investigación. La Tabla 3.1 resume estos objetivos y sus argumentos principales.

Los objetivos `find` y `count` requieren distinguir explícitamente si la entidad considerada corresponde a una persona o a un objeto. Para ello se utiliza el argumento `kind`. Por ejemplo:

Tabla 3.1: Vocabulario de objetivos semánticos utilizado en la representación canónica.

Objetivo	Propósito	Argumentos principales
go	Desplazarse hacia una ubicación	ubicación
find	Localizar una persona u objeto	objetivo, kind , gesto, pose, vestimenta o propiedad
take	Tomar un objeto	objeto
deliver	Entregar un objeto	objeto, destinatario
place	Colocar un objeto en un destino	objeto, destino y relación espacial
guide	Guiar a una persona	persona, destino
follow	Seguir a una persona	persona, destino opcional
count	Contar personas u objetos	objetivo, kind y atributos opcionales
tell	Comunicar información	información o propiedad
save	Conservar información obtenida durante una interacción	nombre, pose o gesto
answer_question	Atender una pregunta	—
greet	Saludar a una persona	persona

```
find(Robin, kind=person)
find(apple, kind=object)
count(person, kind=person)
count(drinks, kind=object)
```

Otros objetivos tienen un tipo de entidad implícito. Por ejemplo, **take**, **deliver** y **place** operan sobre objetos, mientras que **guide**, **follow** y **greet** operan sobre personas.

Los objetivos pueden incluir además calificadores relevantes presentes en la instrucción. Entre ellos se encuentran gestos, poses, vestimenta, características de objetos y relaciones espaciales. Por ejemplo:

```
find(person, kind=person, gesture='waving person')
find(person, kind=person, pose='sitting person')
find(person, kind=person, wearing='red shirt')
find(drink, kind=object, property=largest)
guide(Robin, to=kitchen)
place(apple, on=table)
```

La representación canónica busca conservar la información necesaria para describir la intención de la orden, eliminando al mismo tiempo las diferencias lingüísticas que no modifican su significado dentro del dominio.

La forma concreta utilizada para serializar y transmitir esta secuencia entre componentes pertenece a la implementación y se describe posteriormente.

Fidelidad de la representación semántica

Una secuencia formada únicamente por objetivos admitidos no constituye necesariamente una interpretación correcta.

Considérese nuevamente:

“Go to the kitchen and find Robin.”

La representación esperada es:

```
go(kitchen)
find(Robin, kind=person)
```

Una salida como:

```
find(Robin, kind=person)
```

utiliza un objetivo válido, pero omite la navegación solicitada.

Por otra parte:

```
go(kitchen)
find(Robin, kind=person)
greet(Robin)
```

incorpora una intención que no forma parte de la orden original.

Por tanto, una interpretación correcta debe:

- conservar las subtareas solicitadas;
- evitar objetivos que no se deriven de la instrucción;
- conservar las entidades y sus tipos;
- mantener las relaciones relevantes entre las entidades;
- preservar el orden lógico necesario entre las subtareas.

Cuando se dispone de una representación de referencia G_{ref} , el caso ideal corresponde a que la representación producida, G_{pred} , coincida con ella:

$$G_{\text{pred}} = G_{\text{ref}}.$$

Esta condición es estricta y puede descomponerse posteriormente en criterios más específicos para determinar qué elementos de una interpretación fueron correctos o incorrectos.

3.3.5. Objetivos semánticos y acciones ejecutables

Los objetivos canónicos describen **qué** debe conseguir el robot, pero no necesariamente especifican todas las operaciones concretas requeridas para hacerlo.

Se denota mediante $\mathcal{A}$ el repertorio de acciones disponibles para el sistema de ejecución. Entre ellas se encuentran acciones de navegación, interacción, búsqueda, percepción y manipulación, como `go_to`, `say`, `listen`, `find_person`, `find_object`, `approach`, `take`, `find_space`, `drop` y `give`.

El vocabulario de objetivos y el repertorio de acciones representan niveles diferentes:

$$\boxed{\mathcal{G} \neq \mathcal{A}} \quad (3.8)$$

Por ejemplo, el objetivo:

`deliver(apple, to=Robin)`

representa una intención de alto nivel. Para realizarla pueden ser necesarias acciones concretas destinadas a localizar al destinatario, aproximarse a él y entregar el objeto.

Conceptualmente:

$$\text{deliver} \rightarrow \text{find_person} \rightarrow \text{approach} \rightarrow \text{give}.$$

De manera similar:

`place(apple, on=table)`

puede requerir identificar primero un espacio adecuado y posteriormente liberar el objeto mediante las acciones correspondientes.

El plan resultante se representa mediante una secuencia:

$$P = (a_1, a_2, \dots, a_m),$$

donde cada a_i corresponde a una acción perteneciente al repertorio $\mathcal{A}$.

Un objetivo puede producir una o varias acciones. Por ello, la planificación puede entenderse simplemente como la transformación de la secuencia de objetivos G en un plan P compatible con las capacidades disponibles:

$$G \rightarrow P.$$

Esta distinción permite evaluar por separado si una orden fue interpretada correctamente y si los objetivos obtenidos fueron transformados posteriormente en un plan adecuado.

3.3.6. Dependencias y contexto entre objetivos

La secuencia G no puede tratarse como una colección de objetivos independientes. Algunas subtareas dependen de otras, reutilizan entidades identificadas previamente o establecen condiciones necesarias para objetivos posteriores.

Por esta razón, G se considera una **secuencia ordenada**.

Dependencias de precedencia

Algunos objetivos requieren que una subtarea anterior haya sido realizada.

Por ejemplo:

```
take(apple)
deliver(apple, to=Robin)
```

requiere adquirir el objeto antes de intentar entregarlo. Esta relación se representa mediante:

$$\text{take}(o) \prec \text{deliver}(o, p), \quad (3.9)$$

donde $\prec$ indica que el objetivo situado a la izquierda debe preceder al situado a la derecha.

Una secuencia que invirtiera esta relación podría contener objetivos individualmente válidos, pero no representaría una tarea coherente.

Dependencias referenciales

Algunas instrucciones introducen entidades que posteriormente deben ser reutilizadas.

Considérese:

```
find(person, kind=person, gesture='waving person')  
guide(person, to=kitchen)
```

La persona utilizada en el segundo objetivo no representa una persona arbitraria. Se refiere a la entidad identificada por la búsqueda anterior.

Este tipo de relación permite representar instrucciones que contienen pronombres, referencias genéricas o descripciones cuyo referente se obtiene durante el desarrollo de la tarea.

Dependencias espaciales y de estado

Algunos objetivos establecen un contexto que debe ser considerado por las subtareas posteriores.

Por ejemplo:

```
go(kitchen)  
find(apple, kind=object)
```

indica que la búsqueda del objeto debe realizarse después de desplazarse hacia la cocina. El objetivo de navegación establece así el contexto espacial esperado para la búsqueda posterior.

De manera similar:

```
take(apple)  
deliver(apple, to=Robin)
```

establece una condición intermedia: después de tomar la manzana, el plan puede considerar que el robot dispone del objeto para realizar la entrega.

Estas condiciones representan el **estado esperado durante la planificación**. No constituyen una confirmación de que las acciones hayan sido ejecutadas correctamente en el mundo físico. Esta diferencia se retoma posteriormente al distinguir entre planificación y ejecución.

Orden lingüístico y orden semántico

Finalmente, debe distinguirse el orden en el que las palabras aparecen dentro de una oración del orden lógico requerido para realizar la tarea.

Por ejemplo:

“Bring me the apple from the kitchen.”

expresa directamente una intención de entrega, pero para realizarla puede ser necesario construir la siguiente secuencia:

```
go(kitchen)
find(apple, kind=object)
take(apple)
deliver(apple, to=me)
```

El orden lógico puede resumirse como:

$\text{go} \prec \text{find} \prec \text{take} \prec \text{deliver}.$

La interpretación no consiste, por tanto, en reproducir el orden superficial de las palabras, sino en recuperar las relaciones necesarias entre las subtareas.

Esta propiedad resulta especialmente relevante en instrucciones compactas, en las que algunas operaciones requeridas para cumplir la intención pueden no aparecer expresadas como verbos independientes.

En conjunto, la representación G debe conservar tanto la intención de la orden como las entidades, relaciones y dependencias necesarias para transformarla posteriormente en un plan coherente. De esta manera, la formalización de la presente sección mantiene como eje principal la relación:

$$\boxed{x \rightarrow G \rightarrow P.}$$

3.4. Arquitecturas conceptuales evaluadas

A partir de la formalización anteriormente presentada en la Sección 3.3, el problema estudiado puede resumirse mediante dos etapas principales:

$$x \rightarrow G$$

$$G \rightarrow P,$$

donde una instrucción en lenguaje natural x se transforma primero en una secuencia de objetivos semánticos G y posteriormente en un plan de acciones P .

Las dos arquitecturas consideradas en esta investigación resuelven este mismo problema, pero distribuyen de manera diferente las responsabilidades de interpretación y planificación.

En la arquitectura Qwen–CLIPS, ambas funciones se encuentran separadas: Qwen realiza la interpretación semántica y CLIPS transforma posteriormente los objetivos obtenidos en acciones ejecutables.

En la arquitectura GPT directo, un mismo modelo realiza tanto la interpretación de la orden como la generación del plan.

Esta diferencia constituye el eje principal de la comparación. Por tanto, el objetivo no consiste únicamente en comparar dos modelos de lenguaje, sino en contrastar dos formas distintas de organizar el proceso que relaciona lenguaje natural, objetivos semánticos y acciones ejecutables.

3.4.1. Base común de las arquitecturas evaluadas

Para que la comparación sea significativa, ambas arquitecturas operan sobre una misma definición del problema y comparten los elementos conceptuales establecidos en la sección anterior.

En particular, ambas utilizan:

- el mismo conocimiento del dominio K ;
- el mismo vocabulario de objetivos semánticos $\mathcal{G}$;
- el mismo repertorio de acciones ejecutables $\mathcal{A}$;
- las mismas familias de tareas consideradas para la evaluación;
- una representación común de referencia para determinar qué objetivos corresponden a cada instrucción.

En consecuencia, ambas arquitecturas deben interpretar órdenes pertenecientes al mismo dominio y producir planes compatibles con las mismas capacidades del robot.

Sin embargo, utilizar el mismo dominio no significa que el conocimiento necesario para resolver la tarea se encuentre distribuido de la misma manera.

En Qwen–CLIPS, Qwen debe determinar únicamente qué solicita el usuario. El conocimiento necesario para convertir posteriormente esos objetivos en acciones se encuentra representado mediante reglas dentro de CLIPS.

En cambio, GPT directo debe disponer durante una misma inferencia tanto del conocimiento necesario para interpretar la orden como de las restricciones necesarias para generar el plan.

Por tanto, la comparación se realiza sobre un mismo problema y un mismo resultado esperado, aunque cada arquitectura distribuya de manera diferente el conocimiento utilizado para alcanzarlo.

3.4.2. Arquitectura Qwen–CLIPS

La primera arquitectura adopta una separación explícita entre interpretación semántica y planificación.

Su funcionamiento general puede resumirse mediante:

$$\boxed{x \xrightarrow{\text{Qwen}} G \xrightarrow{\text{CLIPS}} P} \quad (3.10)$$

Qwen actúa como intérprete semántico. Su responsabilidad consiste en recibir la instrucción en lenguaje natural y producir una secuencia ordenada de objetivos pertenecientes al vocabulario definido en $\mathcal{G}$.

Por ejemplo, una instrucción como:

“Bring me an apple from the kitchen.”

puede ser interpretada como:

```
go(kitchen)
find(apple, kind=object)
take(apple)
deliver(apple, to=me)
```

Qwen no determina directamente todas las acciones que deberá ejecutar el robot. Su función termina en la construcción de esta representación semántica.

Antes de ser procesados por CLIPS, los objetivos obtenidos son sometidos a comprobaciones deterministas destinadas a verificar que utilizan la estructura, los tipos de objetivos y las entidades admitidas por el dominio. Los detalles de estas comprobaciones

y de la representación utilizada para intercambiar la información entre componentes se presentan posteriormente en el capítulo de implementación.

La Figura 3.2 resume el flujo conceptual de esta arquitectura.

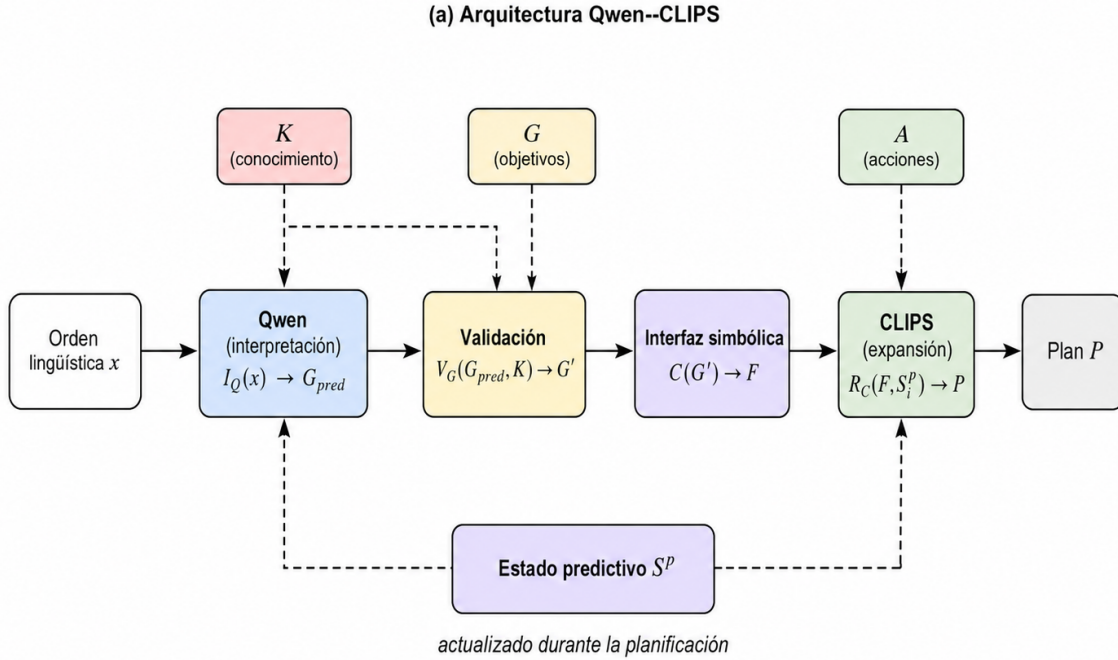


Figura 3.2: Arquitectura conceptual Qwen–CLIPS.

Una vez aceptada la representación semántica, CLIPS se encarga de transformar los objetivos en acciones pertenecientes al repertorio $\mathcal{A}$.

En esta arquitectura, CLIPS funciona como un expansor de objetivos basado en reglas de producción. Su función no consiste en interpretar nuevamente la orden original ni en realizar una búsqueda libre entre planes posibles. Cada objetivo activa conocimiento explícito que determina las acciones necesarias para atenderlo.

Por ejemplo, un objetivo de entrega:

```
deliver(apple, to=Robin)
```

puede expandirse en acciones destinadas a localizar al destinatario, aproximarse a él y entregar finalmente el objeto.

De forma similar, un objetivo de colocación puede requerir identificar primero un espacio adecuado antes de liberar el objeto.

La expansión conserva el orden y las relaciones presentes en G , de manera que las dependencias de precedencia, referencia y contexto descritas en la Sección 3.3.6 puedan ser consideradas durante la generación del plan.

Estado esperado durante la planificación

La generación del plan puede depender de los resultados esperados de objetivos anteriores.

Por ejemplo, después de planificar una navegación hacia la cocina, las subtarear posteriores pueden considerar la cocina como la ubicación esperada del robot. De manera similar, después de planificar la adquisición de un objeto, una tarea posterior de entrega puede asumir que dicho objeto se encuentra en posesión del robot.

Esta información constituye un **estado esperado de planificación**. Su propósito es mantener coherencia entre las acciones que forman el plan.

Sin embargo, dicho estado no representa una observación confirmada del mundo físico. Que el plan considere que el robot ha llegado a una habitación o posee un objeto no garantiza que la acción correspondiente haya sido completada correctamente durante la ejecución.

Esta diferencia resulta importante para separar los errores producidos durante la planificación de aquellos que pueden aparecer posteriormente durante la operación física del robot.

Separación y propagación de errores

La división de responsabilidades entre Qwen y CLIPS permite distinguir con claridad dos posibles fuentes de error.

La primera corresponde a la interpretación. Si Qwen produce una secuencia de objetivos diferente de la esperada, CLIPS puede generar posteriormente un plan perfectamente coherente con dicha secuencia, aunque ésta represente de manera incorrecta la intención original.

En otras palabras:

plan coherente $\nRightarrow$ interpretación correcta

La segunda fuente de error corresponde a la expansión simbólica. Incluso cuando los objetivos representan correctamente la orden, las reglas disponibles deben ser capaces de transformarlos en una secuencia adecuada de acciones.

Esta separación permite evaluar de manera independiente la calidad de la interpretación y la calidad de la planificación, aspecto que posteriormente se aprovecha mediante una condición experimental de control.

3.4.3. Arquitectura de planificación directa con GPT

La segunda arquitectura concentra en un mismo modelo las funciones de interpretación semántica y generación del plan.

Su comportamiento principal puede resumirse mediante:

$$\boxed{x \xrightarrow{\text{GPT}} (G, P)} \quad (3.11)$$

Para una instrucción ejecutable, GPT produce en una misma respuesta tanto la secuencia de objetivos semánticos G como el plan de acciones P .

La conservación explícita de ambos elementos resulta importante. Aunque hayan sido generados por el mismo modelo, G representa la interpretación realizada sobre la orden, mientras que P representa la secuencia de acciones propuesta para atenderla.

De esta manera, ambos resultados pueden ser analizados posteriormente de forma independiente.

La Figura 3.3 presenta el flujo conceptual de esta arquitectura.

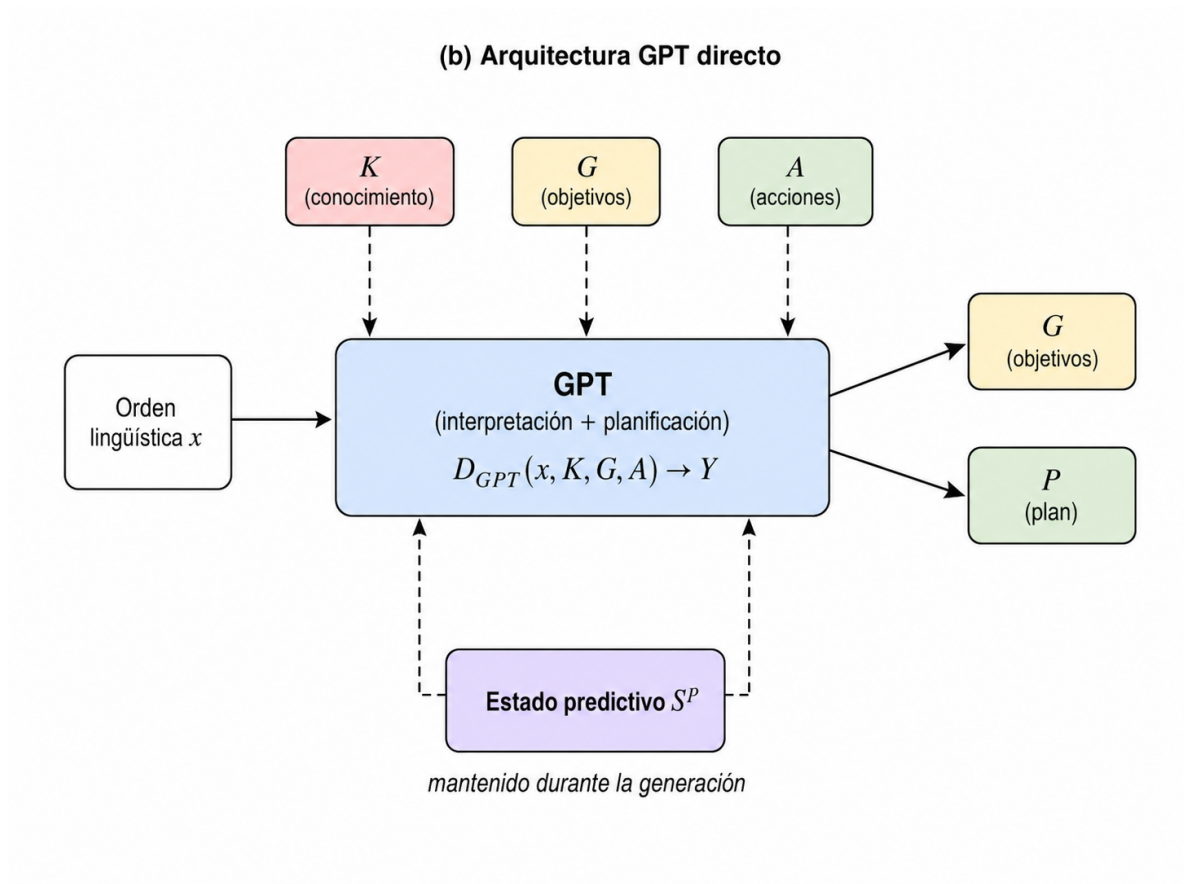


Figura 3.3: Arquitectura conceptual del planificador GPT directo.

Una diferencia fundamental respecto a Qwen-CLIPS es que los objetivos producidos por GPT no se envían posteriormente a CLIPS. El mismo modelo que interpreta la instrucción determina también las acciones que forman el plan.

Por tanto, el flujo no corresponde a:

$$G_{\text{GPT}} \rightarrow \text{CLIPS} \rightarrow P,$$

sino que tanto G como P constituyen productos de la misma inferencia.

Tipos de respuesta

La arquitectura no obliga al modelo a producir un plan para cualquier instrucción recibida. Dependiendo de la información disponible y de la compatibilidad de la orden con el dominio, se contemplan tres tipos de respuesta:

- **executable:** la instrucción contiene información suficiente y puede representarse mediante objetivos y acciones soportadas. En este caso se generan G y P .
- **clarification:** existe información faltante o una ambigüedad que puede resolverse mediante una pregunta breve al usuario. En este caso no se genera un plan hasta resolver dicha ambigüedad.
- **rejected:** la instrucción se encuentra fuera del dominio, requiere una capacidad no disponible o contradice las restricciones establecidas. En este caso tampoco se genera un plan.

Esta clasificación permite evitar que el modelo genere arbitrariamente una secuencia de acciones cuando no dispone de información suficiente para interpretar la orden.

Planificación directa bajo restricciones

El término *planificación directa* indica que GPT produce las acciones del plan sin delegar posteriormente la expansión de los objetivos a CLIPS.

Esto no significa que el modelo pueda generar cualquier comportamiento.

La generación se encuentra restringida mediante información sobre:

- las entidades admitidas en el dominio;
- los tipos de objetivos pertenecientes a $\mathcal{G}$;
- las acciones disponibles en $\mathcal{A}$;
- las relaciones y dependencias necesarias entre subtareas;
- las restricciones que determinan qué combinaciones pueden constituir un plan válido.

Así, una intención semántica como **deliver** no debe utilizarse directamente como una acción ejecutable si no pertenece al repertorio $\mathcal{A}$. El modelo debe generar las acciones concretas necesarias para realizar dicha intención.

El conocimiento que en Qwen-CLIPS se encuentra expresado mediante reglas de producción debe encontrarse disponible para GPT mediante las instrucciones y restricciones proporcionadas durante la generación.

Esta diferencia no modifica el dominio de la comparación, sino la forma en que cada arquitectura representa y utiliza el conocimiento necesario para construir el plan.

Estado esperado durante la generación

Al igual que la arquitectura Qwen-CLIPS, GPT debe mantener coherencia entre las acciones que genera.

Por ejemplo, una acción de entrega debe aparecer después de las acciones necesarias para disponer del objeto y localizar al destinatario. De manera similar, una búsqueda puede depender de una navegación previamente incluida en el plan.

Para ello, el modelo debe razonar sobre los resultados esperados de las acciones anteriores durante la construcción de la secuencia.

Este estado es únicamente predictivo. No corresponde a información obtenida posteriormente de los sensores del robot ni confirma que las acciones hayan sido realizadas correctamente en el mundo físico.

La diferencia entre ambas arquitecturas radica principalmente en el mecanismo utilizado para conservar esta coherencia: Qwen-CLIPS utiliza conocimiento simbólico y reglas explícitas, mientras que GPT debe mantenerla durante la generación condicionada por las instrucciones y restricciones proporcionadas.

Separación posterior de interpretación y planificación

Aunque GPT realice interpretación y planificación durante una misma inferencia, la conservación de G y P permite analizar posteriormente ambas funciones por separado.

Si los objetivos producidos no corresponden con los objetivos de referencia, el error puede atribuirse principalmente a la interpretación de la orden.

En cambio, si G representa correctamente la instrucción pero P omite acciones necesarias, utiliza acciones no soportadas o viola dependencias de la tarea, puede identificarse un error de planificación.

A diferencia de Qwen-CLIPS, esta separación no corresponde a dos componentes independientes del sistema. Se obtiene mediante el análisis de los dos artefactos generados por el mismo modelo.

3.4.4. Comparación conceptual de las arquitecturas

La diferencia esencial entre las arquitecturas puede resumirse mediante:

$$\begin{array}{l} \text{Qwen-CLIPS: } x \rightarrow G \rightarrow P \\ \text{GPT directo: } x \rightarrow (G, P) \end{array} \quad (3.12)$$

En Qwen-CLIPS, la interpretación y la planificación constituyen etapas explícitamente separadas: Qwen produce la representación semántica y CLIPS transforma posteriormente dicha representación en acciones.

En GPT directo, ambas funciones se concentran en una misma inferencia, aunque los objetivos y el plan se conservan como resultados diferenciados para facilitar su análisis.

La Tabla 3.2 resume las principales características de ambas estrategias.

Tabla 3.2: Comparación conceptual entre las arquitecturas evaluadas.

Característica	Qwen-CLIPS	GPT directo
Interpretación semántica	Qwen	GPT
Generación del plan	CLIPS	GPT
Representación explícita de G	Sí	Sí
Generación de P	Reglas de producción	Generación condicionada del modelo
Conocimiento para expandir los objetivos	Representado explícitamente mediante reglas	Proporcionado mediante contexto y restricciones
Separación entre interpretación y planificación	Explícita entre componentes	Integrada en una misma inferencia
Estado durante la planificación	Predictivo y simbólico	Predictivo durante la generación
Dependencia de un servicio externo de terceros	No	Sí
Identificación de errores	Separación directa entre etapas	Separación posterior mediante G y P
Variabilidad del resultado	Depende de los objetivos, reglas y condiciones iniciales	Puede presentar variación entre inferencias

El carácter basado en reglas de CLIPS no implica que cualquier plan producido sea necesariamente correcto ni garantiza el éxito de su ejecución física. De manera similar, restringir la estructura de la respuesta de GPT no garantiza que la interpretación o el plan generado sean semánticamente correctos.

Por ello, en ambas arquitecturas resulta necesario evaluar por separado la representación semántica G y el plan P .

La comparación realizada en esta investigación se centra, por tanto, en dos estrategias conceptualmente diferentes:

- **Qwen-CLIPS:** separación explícita entre interpretación lingüística y expansión simbólica del plan.
- **GPT directo:** integración de interpretación y planificación en un mismo modelo.

Esta diferencia permite analizar posteriormente no sólo qué arquitectura produce una mayor proporción de resultados correctos, sino también dónde se originan los erro-

res, qué grado de consistencia presenta cada estrategia y qué recursos requiere para transformar una instrucción en un plan compatible con las capacidades del robot.

3.5. Criterios conceptuales de validez

La generación de una salida correctamente estructurada no implica, por sí misma, que la interpretación de una orden o el plan correspondiente sean correctos.

Por esta razón, en esta investigación se distinguen diferentes niveles de validez. El propósito de esta separación es identificar si un error se origina durante la interpretación de la instrucción, durante la generación del plan o posteriormente durante su ejecución física.

De manera general, el proceso puede representarse como:

$$\boxed{x \xrightarrow{\text{interpretación}} G \xrightarrow{\text{planificación}} P \xrightarrow{\text{ejecución}} E} \quad (3.13)$$

donde x representa la instrucción en lenguaje natural, G la secuencia de objetivos semánticos, P el plan generado y E el resultado observado durante la ejecución.

Cada una de estas etapas debe evaluarse de manera independiente.

3.5.1. Niveles de validez

Para analizar las salidas producidas por las arquitecturas evaluadas se consideran los siguientes niveles:

1. **Validez sintáctica:** la salida puede ser procesada por el sistema y respeta la sintaxis esperada.
2. **Validez estructural:** los objetivos, acciones, argumentos y tipos utilizados pertenecen a las estructuras admitidas por el sistema.
3. **Validez semántica:** la representación G conserva la intención expresada en la instrucción original.
4. **Validez del plan:** la secuencia P constituye una realización coherente de los objetivos representados en G .
5. **Éxito de ejecución:** el robot consigue realizar físicamente las acciones planificadas y alcanzar el resultado esperado.

Estos niveles no deben considerarse equivalentes.

Una salida puede ser sintáctica y estructuralmente válida y, aun así, representar

incorrectamente la intención del usuario. De manera similar, un plan puede ser conceptualmente correcto pero fallar durante la ejecución debido a condiciones externas al proceso de planificación.

Esta separación permite evitar que todos los fallos observados sean agrupados bajo una única categoría de “plan incorrecto”.

3.5.2. Validez de la interpretación semántica

La interpretación se considera válida cuando la secuencia de objetivos G representa fielmente la tarea expresada en la instrucción x dentro del dominio considerado.

Los requisitos de fidelidad fueron introducidos en la Sección 3.3.4. Para la evaluación conceptual se consideran principalmente los siguientes criterios:

1. **Cobertura de las subtareas:** deben conservarse todos los objetivos necesarios para representar la tarea, incluyendo aquellas operaciones implícitas que resulten necesarias dentro del dominio.
2. **Ausencia de intenciones no solicitadas:** la representación no debe introducir objetivos adicionales que modifiquen la intención original de la orden.
3. **Exactitud de entidades y tipos:** las personas, objetos, categorías y ubicaciones utilizadas deben corresponder con las referencias presentes en la instrucción y ser compatibles con el conocimiento K .
4. **Conservación de relaciones y dependencias:** deben mantenerse las relaciones de precedencia, referencia y contexto necesarias entre los objetivos.

Cuando se dispone de una representación de referencia G_{ref} , el caso ideal corresponde a que la representación producida G_{pred} coincida con ella:

$$G_{\text{pred}} = G_{\text{ref}}.$$

Esta condición representa una correspondencia estricta entre la interpretación obtenida y la esperada. Durante la evaluación puede complementarse con criterios más específicos para identificar omisiones, adiciones, errores de entidad, relaciones incorrectas u otros tipos de discrepancia.

Consistencia ante reformulaciones

La variabilidad lingüística introduce un criterio adicional de validez.

Como se definió en la Sección 3.3.2, dos formulaciones diferentes pueden representar una misma tarea. En ese caso, idealmente ambas deben producir la misma representación semántica:

$$x_1 \sim x_2 \quad \Rightarrow \quad G_1 = G_2.$$

Esta propiedad resulta especialmente relevante en GPSR, donde una misma instrucción puede ser reformulada sin modificar sus objetivos.

Por tanto, la capacidad de interpretar correctamente una orden no se evalúa únicamente mediante una formulación aislada, sino también mediante la consistencia del sistema ante diferentes expresiones de una misma intención.

3.5.3. Validez y consistencia del plan

Una vez obtenida la representación semántica G , debe determinarse si el plan P constituye una realización coherente de dichos objetivos.

Un plan se considera conceptualmente válido cuando cumple los siguientes criterios:

1. **Acciones soportadas:** todas las acciones utilizadas pertenecen al repertorio $\mathcal{A}$ disponible para el sistema de ejecución.
2. **Parámetros válidos:** cada acción utiliza argumentos compatibles con su función y con las entidades del dominio.
3. **Respeto de dependencias:** la secuencia conserva el orden y las relaciones necesarias para realizar la tarea.
4. **Cobertura de los objetivos:** las acciones generadas permiten atender los objetivos contenidos en G sin omitir partes necesarias de la tarea.
5. **Coherencia con el estado esperado:** las acciones posteriores son compatibles con los resultados esperados de las acciones previamente incluidas en el plan.

Por ejemplo, una acción de entrega de un objeto debe aparecer después de las acciones necesarias para obtener dicho objeto y localizar al destinatario.

De manera similar, una acción de aproximación a una persona requiere que dicha persona haya sido identificada previamente.

Por tanto, una secuencia puede estar formada únicamente por acciones válidas y, aun así, constituir un plan incorrecto si las acciones aparecen en un orden inadecuado, utilizan parámetros incompatibles o no cubren completamente los objetivos solicitados.

3.5.4. Validez de la respuesta del sistema

Los criterios anteriores se aplican principalmente a órdenes que pueden ser interpretadas y planificadas dentro del dominio.

Sin embargo, la arquitectura GPT directo contempla también situaciones en las que no resulta apropiado generar inmediatamente un plan.

Como se describió en la Sección 3.4.3, se consideran tres tipos de respuesta:

- **executable:** la orden contiene información suficiente y puede ser atendida mediante los objetivos y acciones disponibles.
- **clarification:** la orden contiene una ambigüedad o información faltante que puede resolverse solicitando una aclaración al usuario.
- **rejected:** la solicitud se encuentra fuera del dominio, requiere una capacidad no disponible o contradice las restricciones establecidas.

Por ejemplo:

“Bring it to her.”

puede requerir una aclaración si no existe suficiente contexto para determinar qué representan *it* y *her*.

En este tipo de situaciones, generar arbitrariamente un plan no debe considerarse una respuesta correcta.

Por tanto, la evaluación debe considerar también si el sistema selecciona adecuadamente entre generar un plan, solicitar información adicional o rechazar la instrucción.

3.5.5. Condición de control para la etapa simbólica

La separación explícita entre interpretación y planificación de la arquitectura Qwen-CLIPS permite introducir una condición experimental de control destinada a aislar el comportamiento del componente simbólico.

En esta condición, los objetivos de referencia se proporcionan directamente a CLIPS:

$$\boxed{G_{\text{ref}} \xrightarrow{\text{CLIPS}} P_{\text{CLIPS}}} \quad (3.14)$$

Esta condición no constituye una tercera arquitectura de planificación. Su propósito es evaluar únicamente la transformación:

$$G \rightarrow P,$$

eliminando temporalmente los posibles errores producidos durante:

$$x \rightarrow G.$$

Si CLIPS genera un plan correcto cuando recibe G_{ref} , pero el flujo completo Qwen-CLIPS falla para la misma instrucción, el error puede encontrarse en la interpretación producida por Qwen.

En cambio, si el plan generado a partir de G_{ref} también es incorrecto, el problema corresponde a la expansión simbólica, la cobertura de las reglas o la generación del plan.

Esta condición permite distinguir experimentalmente entre dos tipos principales de error:

- **error de interpretación:** la secuencia G no representa correctamente la instrucción;
- **error de planificación:** la representación G es correcta, pero el plan P no constituye una expansión adecuada.

Esta separación será utilizada posteriormente para clasificar los resultados obtenidos durante la evaluación.

3.5.6. Planificación y ejecución física

El último nivel de análisis corresponde a la diferencia entre generar un plan correcto y completar satisfactoriamente su ejecución sobre el robot.

La relación fundamental puede expresarse de manera sencilla como:

$$\boxed{\text{plan correcto} \not\Rightarrow \text{ejecución exitosa}} \quad (3.15)$$

Un plan puede representar correctamente la intención de la instrucción y contener una secuencia coherente de acciones, pero su ejecución puede fallar por factores externos al proceso de planificación.

Entre ellos se encuentran, por ejemplo:

- fallos durante la percepción de personas u objetos;
- obstáculos o cambios en el entorno durante la navegación;
- dificultades durante la manipulación de objetos;
- cambios en la posición de personas u objetos;
- problemas durante la interacción humano-robot.

Por ejemplo, el plan puede indicar correctamente que el robot debe tomar una manzana, pero la ejecución puede fallar si el objeto no es detectado o si el manipulador no consigue sujetarlo.

De manera similar, una acción de navegación puede estar correctamente planificada y, aun así, no completarse debido a obstáculos o modificaciones del entorno.

Por esta razón, el estado esperado utilizado durante la planificación no debe confundirse con el estado real observado durante la ejecución.

La evaluación de las arquitecturas debe distinguir, por tanto, entre:

1. la correcta interpretación de la instrucción;
2. la generación de un plan coherente con dicha interpretación;
3. el resultado obtenido al intentar ejecutar físicamente ese plan.

Esta separación permite analizar los resultados sin atribuir automáticamente a la interpretación o a la planificación fallos originados en otros componentes del robot.

Los criterios establecidos en esta sección definen conceptualmente qué se considera una interpretación correcta, un plan válido y una ejecución exitosa. Las métricas concretas, procedimientos experimentales y criterios utilizados para medir estas propiedades se presentan posteriormente en el capítulo de validación y experimentación.

Capítulo 4

Metodología y diseño de las arquitecturas evaluadas

4.1. Diseño metodológico de la investigación

4.1.1. Problema experimental

4.1.2. Variables y condiciones de comparación

4.1.3. Estrategia general de evaluación

4.2. Construcción del dominio experimental

4.2.1. Fuentes de conocimiento GPSR

4.2.2. Catálogos de personas, objetos y ubicaciones

4.2.3. Familias de tareas

4.2.4. Representación de los objetivos de referencia

4.3. Construcción del conjunto de datos

4.3.1. Generación de órdenes

4.3.2. Generación de objetivos canónicos

4.3.3. Variación lingüística

4.3.4. Balance y deduplicación

4.3.5. División de entrenamiento y evaluación

Capítulo 5

Implementación

5.1. Plataforma experimental

5.1.1. Robot Justina

5.1.2. Plataforma de cómputo

5.1.3. Entorno de software y ROS 2

5.2. Implementación del conjunto de datos

5.2.1. Generador de órdenes

5.2.2. Catálogos y CompetitionTemplate

5.2.3. Validadores

5.2.4. Dataset final

5.3. Entrenamiento e inferencia de Qwen2.5-7B

5.3.1. Preparación del modelo

5.3.2. Configuración QLoRA

5.3.3. Hiperparámetros de entrenamiento

5.3.4. Selección del checkpoint

5.3.5. Inferencia

5.3.6. Despliegue del modelo

Capítulo 6

Validación y experimentación

6.1. Diseño experimental

6.1.1. Conjunto de prueba

6.1.2. Procedimiento experimental

6.1.3. Repeticiones y control de variabilidad

6.2. Condiciones experimentales

6.2.1. Arquitectura Qwen–CLIPS

6.2.2. Arquitectura GPT directo

6.2.3. Condición de control con objetivos de referencia y CLIPS

6.3. Evaluación de la interpretación semántica

6.3.1. Exactitud de objetivos

6.3.2. Validez estructural

6.3.3. Exactitud de entidades y parámetros

6.3.4. Dependencias entre objetivos

6.4. Evaluación de la planificación

6.4.1. Acciones válidas

6.4.2. Consistencia del plan

Capítulo 7

Resultados

7.1. Resultados de la especialización de Qwen

7.1.1. Comportamiento durante entrenamiento

7.1.2. Evaluación del modelo seleccionado

7.2. Resultados de interpretación semántica

7.2.1. Qwen

7.2.2. GPT

7.2.3. Comparación

7.3. Resultados de planificación

7.3.1. Qwen–CLIPS

7.3.2. GPT directo

7.3.3. Condición de control CLIPS

7.3.4. Comparación

7.4. Análisis de errores

7.4.1. Errores de interpretación

7.4.2. Errores de planificación

7.4.3. Casos representativos

Capítulo 8

Discusión

8.1. Interpretación de los resultados respecto a la hipótesis

8.2. Interpretación semántica

8.2.1. Especialización de Qwen

8.2.2. Capacidad de GPT

8.2.3. Efecto de la variabilidad lingüística

8.3. Planificación de acciones

8.3.1. Expansión determinista mediante CLIPS

8.3.2. Planificación generativa mediante GPT

8.3.3. Separación entre errores de interpretación y planificación

8.4. Comparación Qwen–CLIPS frente a GPT directo

8.4.1. Exactitud

8.4.2. Consistencia

8.4.3. Latencia

Capítulo 9

Conclusiones y trabajo futuro

- 9.1. Conclusiones principales
- 9.2. Cumplimiento de los objetivos
- 9.3. Evaluación de la hipótesis
- 9.4. Contribuciones de la investigación
 - 9.4.1. Representación canónica de órdenes
 - 9.4.2. Arquitectura Qwen–CLIPS
 - 9.4.3. Comparación con planificación directa mediante GPT
 - 9.4.4. Metodología para separar errores de interpretación y planificación
- 9.5. Trabajo futuro

Capítulo 10

Conclusiones y Trabajo Futuro

10.1. Conclusiones Principales

10.2. Propuestas de Trabajo Futuro

- Fine-tuning de un LLM de código abierto para dominio específico
- Mejora de los mecanismos de seguridad y verificación
- Exploración de arquitecturas de integración más profundas

Referencias

- [1] R. R. Murphy, *Introduction to AI Robotics*. MIT Press, 2nd ed., 2019.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 4th ed., 2020.
- [3] J. C. Giarratano and G. D. Riley, *Expert Systems: Principles and Programming*. Thomson Course Technology, 4th ed., 2005.
- [4] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal of Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] L. Shi, X. Chen, Y. Hu, J. Li, H. Zhang, and H. Zhao, “Chatgpt for robotics: Design principles and model abilities,” *Microsoft Research*, 2023.
- [7] NASA Johnson Space Center, “Clips reference manual,” tech. rep., NASA, 1991.
- [8] B. Pell *et al.*, “An autonomous spacecraft agent prototype,” in *First International Conference on Autonomous Agents*, pp. 253–261, 1997.
- [9] J. Savage and otros, “Virbot: A modular architecture for service robots with hybrid planning,” in *RoboCup Symposium*, 2024.
- [10] L. Cheng, Y. Mao, and T. Ward, “Editorial: Neural network models in autonomous robotics,” *Frontiers in Neurorobotics*, 2025.
- [11] Alibaba Cloud Qwen Team, “Qwen official github repository,” 2026.
- [12] OpenAI, “Openai developer documentation and model guides,” 2026.
- [13] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for

- NLP,” in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97 of *Proceedings of Machine Learning Research*, pp. 2790–2799, PMLR, 2019.
- [14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [15] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, pp. 10088–10115, 2023.
- [16] A. d’Avila Garcez, T. R. Besold, L. De Raedt, P. Földiák, P. Hitzler, T. Icard, K.-U. Kühnberger, L. C. Lamb, R. Miikkulainen, and D. L. Silver, “Neural-symbolic learning and reasoning: Contributions and challenges,” in *Knowledge Representation and Reasoning: Integrating Symbolic and Neural Approaches: Papers from the 2015 AAAI Spring Symposium*, pp. 18–21, AAAI Press, 2015.
- [17] T. R. Besold, A. d’Avila Garcez, S. Bader, H. Bowman, P. Domingos, P. Hitzler, K.-U. Kuehnberger, L. C. Lamb, D. Lowd, P. M. V. Lima, L. de Penning, G. Pinkas, H. Poon, and G. Zaverucha, “Neural-symbolic learning and reasoning: A survey and interpretation,” *arXiv preprint arXiv:1711.03902*, 2017.
- [18] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, “LLM+P: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477*, 2023.
- [19] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, *et al.*, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.